QUALCOMM AI RESEARCH (QARE) PREP • EYE-CARE WARM PARCHMENT EDITION

Pillar 3: Computer Vision — Intuitive Study Textbook

2D/3D Convolutions, Receptive Field, Depthwise Separable, Backbones • Detection (IoU/NMS/YOLO/DETR), Segmentation, ViT, CLIP & Diffusion (CV01–CV20)

INTERACTIVE DIRECTORY • CLICK ANY TOPIC TO JUMP

Table of Contents (Mục Lục)

13 Sections Indexed

Day 6: Convolution Arithmetic, FLOPs & Efficient Edge CNNs ..... p. 2

1.1 🌟 [Core — Bread & Butter] The 3 Master Spatial Dimension Formulas (Standard, Dilated & Transposed Convolutions — CV01, CV03) ..... p. 2

1.2 🌟 [Core — Bread & Butter] Exact Parameter & MACs/FLOPs Formulas Across Convolution Variants (CV02, CV04) ..... p. 6

1.3 🌟 [Core — Bread & Butter] Landmark CNN & Edge Architectures: VGG, ResNet, MobileNetV2, GAP & SENet (CV05, CV06, CV07, CV18, CV19) ..... p. 9

Day 7: Receptive Field Recursion, Object Detection & Segmentation ..... p. 13

2.1 🌟 [Core — Bread & Butter] Iterative Receptive Field Calculation (CV01, CV08) ..... p. 13

2.2 🌟 [Core — Bread & Butter] Object Detection Part 1: IoU, GIoU, NMS & Feature Pyramid Networks (CV09, CV10) ..... p. 15

2.3 🌟 [Core — Bread & Butter] Object Detection Part 2: Two-Stage vs. One-Stage, RoI Align (2D Bilinear Interpolation), Focal Loss, DETR & mAP (CV11, CV12) ..... p. 18

2.4 🌟 [Core — Bread & Butter] Semantic, Instance & Panoptic Segmentation (CV13, CV14) ..... p. 22

Day 8: Vision Transformers (ViT, Swin), Multimodal (CLIP) & Diffusion ..... p. 24

3.1 🌟 [Core — Bread & Butter] Vision Transformer (ViT) & Swin Transformer (CV15, CV16) ..... p. 24

3.2 🌟 [Core — Bread & Butter] CLIP (Contrastive Language-Image Pre-training) & Modern Augmentation/SSL (CV17, CV20) ..... p. 27

3.3 🔬 [Core + Deep-Dive] Diffusion Models & Generative Vision: DDPM, DDIM, Latent Diffusion & CFG ..... p. 30

> ⚡ **IMPORTANT**
>
> **How to Use This Guide (Days 6–8: Nov 3 – Nov 5):** Computer Vision questions on the Qualcomm AI Residency Phase 1 MCQ split evenly between **exact tensor, parameter, and receptive-field arithmetic** (where a single off-by-one error in padding, bias counting, or dilation costs the point) and **architectural design reasoning** (why MobileNetV2 removes ReLU in its bottleneck, why RoI Align beats RoI Pooling, why DETR needs no NMS, how ViT interpolates positional embeddings, and how Latent Diffusion & Classifier-Free Guidance work).
>
> Every section in this textbook follows a **4-part learning flow**:
>
> 1. 💡 **Intuition (Plain English First):** What problem does this solve, why was it invented, and what is happening visually? (With **inline single-concept diagrams** right where you read!)
> 2. 📐 **The Math & Every Symbol Explained:** Clean display equations with every variable and matrix shape ($n \times m$, i.e., n rows and m columns) defined explicitly.
> 3. 🔢 **Step-by-Step Worked Toy Example:** Concrete small-number calculations worked out line by line.
> 4. 🎯 **Exam Takeaway & Common Traps:** The 60-second rule of thumb and exact traps tested in `CV01`–`CV20`.

> 💡 **TIP**
>
> 🎯 **80/20 Bread & Butter Priority Map (Focus Here First!):**
>
> - 🌟 **[Core — Bread & Butter] (85% of Exam Marks):** Focus on **Conv2d output size** $\lfloor \frac{I+2P-K_{\text{eff}}}{S} \rfloor + 1$ **& Dilated** $K_{\text{eff}} = D(K-1)+1$, **Standard vs. Depthwise Separable (`DW+PW`) vs. Grouped Conv parameter & MAC counts, ResNet Bottleneck vs. MobileNetV2 Inverted Residual (no ReLU on narrow projection!), GAP parameter reduction** ($H \times W$), **Receptive Field recursion** ($RF_l = RF_{l-1} + (K_{\text{eff},l}-1)J_{l-1}$), **IoU / Dice** ($2\,\text{IoU}/(1+\text{IoU})$) **/ NMS, Focal Loss** $(1-p_t)^\gamma$, **ViT patches** ($L = HW/P^2+1$), and **CLIP** $N \times N$ **contrastive matrix** ($\mathcal{L}_0 = \ln N$).
> - 🔬 **[Optional PhD Deep-Dives] (Collapsible `<details>` Blocks):** ConvNeXt micro-design, DIoU/CIoU equations, RoI Align bilinear algebra, and DDPM/Score-Matching/CFG derivations are tucked inside expandable `<details>` blocks!

Day 6: Convolution Arithmetic, FLOPs & Efficient Edge CNNs

↑ Mục Lục

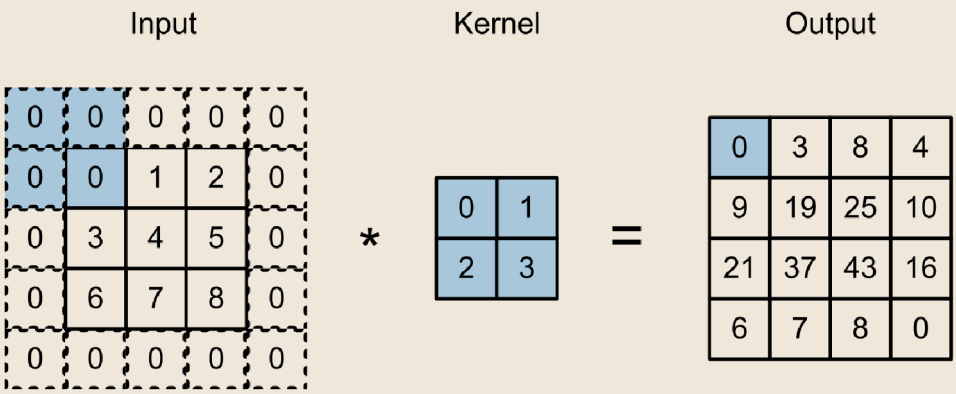
1.1 🌟 [Core — Bread & Butter] The 3 Master Spatial Dimension Formulas (Standard, Dilated & Transposed Convolutions — `CV01`, `CV03`)

💡 Intuition (Plain English First)

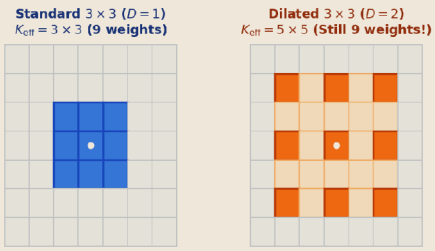
Imagine sliding a small magnifying glass (a **kernel** of size $K \times K$) across an image of height H_{in} and width W_{in}:

- **Padding (P)** adds a border of zeros around the edges of the input image so the kernel can center itself on border pixels without the output shrinking at every layer.

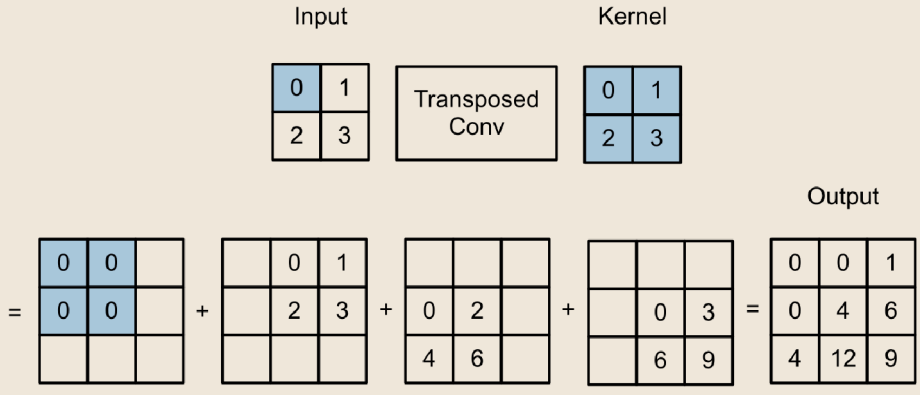
- **Stride (S)** controls how many pixels the magnifying glass jumps after each step. A stride of $S = 1$ slides pixel-by-pixel; a stride of $S = 2$ skips every other pixel, cutting the output height and width roughly in half.
- **Dilation (D, "Atrous Convolution")** pokes empty holes (spaces) between the kernel weights so the filter covers a wider spatial area **without adding any extra learnable parameters**. Standard convolution has $D = 1$ (weights sit right next to each other). When $D = 2$, we insert 1 zero-gap between every adjacent weight, turning a 3×3 weight grid into a 5×5 footprint while still only storing $3 \times 3 = 9$ numbers!
- **Transposed Convolution (`nn.ConvTranspose2d`)** runs the spatial math backward to **upsample** a small feature map into a larger one (e.g., doubling $16 \times 16 \rightarrow 32 \times 32$ in a decoder or generator). However, if the kernel size K is not cleanly divisible by the stride S (for example, $K = 3, S = 2$), some output pixels get hit by more overlapping kernel weights than their neighbors, creating ugly grid-like **checkerboard artifacts**. Modern architectures fix this using **Sub-Pixel Convolution (`nn.PixelShuffle`)**, which generates extra channels via a standard convolution and rearrangements them into higher spatial resolution like laying down mosaic tiles—with zero uneven overlap!



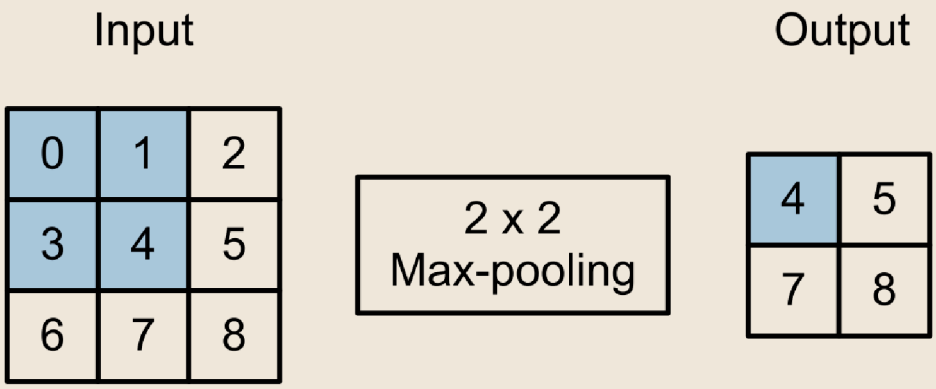
2D Convolution with Zero-Padding



Standard 3x3 vs. Dilated 3x3 (D=2) Convolution Footprint



Transposed Convolution Spatial Upsampling



2D Max Pooling vs. Average Pooling

📐 The Math & Every Symbol Explained

Let an input feature map have spatial dimensions $H_{\text{in}} \times W_{\text{in}}$ (an $n \times m$ spatial grid with $n = H_{\text{in}}$ rows and $m = W_{\text{in}}$ columns). For any single spatial dimension I (representing either H_{in} or W_{in}):

- I: Input spatial size (H_{in} or W_{in}) in pixels.
- O: Output spatial size (H_{out} or W_{out}) in pixels.
- K: Physical kernel size (number of weight taps along one dimension, e.g., $K = 3$ for a 3×3 filter).
- P: Zero-padding added to **each** side of the input (top/bottom or left/right), so total padding along one axis is $2P$.
- S: Stride (step size in pixels between consecutive kernel positions).
- D: Dilation rate (spacing between adjacent kernel elements; $D = 1$ is a standard contiguous kernel, and $D - 1$ is the number of skipped pixels between adjacent weights).
- K_{eff}: Effective kernel footprint on the input grid after accounting for dilation gaps.
- P_{out}: Output padding (`output_padding` in `nn.ConvTranspose2d`, default 0), used only to resolve ambiguity when multiple input sizes map to the same strided output size.
- r: Upscaling factor in Sub-Pixel Convolution (`nn.PixelShuffle(r)`).

1. **Effective Kernel Size (with Dilation $D \geq 1$):**

$$K_{\text{eff}} = K + (K - 1)(D - 1) = D(K - 1) + 1$$

(Why? A kernel of K weights has $K - 1$ gaps between them, and each gap spans D steps instead of 1.)

2. **Standard / Dilated Conv2d & Pooling Output Size:**

$$O = \left\lfloor \frac{I + 2P - K_{\text{eff}}}{S} \right\rfloor + 1 = \left\lfloor \frac{I + 2P - D(K - 1) - 1}{S} \right\rfloor + 1$$

   - **"Same" Padding Condition ($O = I$ when $S = 1$):**

$$I = I + 2P - D(K - 1) \implies P = \frac{D(K - 1)}{2}$$

3. **Transposed Convolution (`nn.ConvTranspose2d`) Output Size:**

$$O = (I - 1) \times S - 2P + D(K - 1) + P_{\text{out}} + 1$$

- For standard settings ($D = 1, P_{\text{out}} = 0$), this simplifies to:

$$O = (I - 1)S - 2P + K$$

4. **Sub-Pixel Convolution (`nn.PixelShuffle(r)`) Tensor Reshape:**

$$\text{Shape}: \quad (B,\, C_{\text{out}} \cdot r^2,\, H,\, W) \longrightarrow (B,\, C_{\text{out}},\, H \cdot r,\, W \cdot r)$$

where B is batch size and C_{out} is the target output channel count.

🔢 Step-by-Step Worked Toy Example

- **Example A (Dilated Convolution Output Size & Parameter Count — `CV01`):** Suppose we feed an input tensor of shape ($C_{\text{in}} = 16$, $H_{\text{in}} = 32$, $W_{\text{in}} = 32$) into a 2D convolution layer with $C_{\text{out}} = 32$ filters, kernel size $K = 3$, stride $S = 1$, padding $P = 1$, dilation $D = 2$, and `bias=False`.
  1. **Step 1 (Compute Effective Kernel Size K_{eff}):**

$$K_{\text{eff}} = D(K - 1) + 1 = 2(3 - 1) + 1 = 5$$

     So our 3×3 kernel spans a 5×5 spatial patch on the input!
  2. **Step 2 (Compute Output Spatial Size O):**

$$O = \left\lfloor \frac{32 + 2(1) - 5}{1} \right\rfloor + 1 = \left\lfloor \frac{29}{1} \right\rfloor + 1 = 30$$

     The output shape is $(32, 30, 30)$. Notice that $P = 1$ did **not** preserve $O = 32$ because dilation expanded K_{eff} to 5! To get "Same" output ($O = 32$) with $K = 3, D = 2, S = 1$, we would need $P = \frac{2(3-1)}{2} = 2$.
  3. **Step 3 (Count Learnable Parameters):** Does dilation $D = 2$ increase the number of weights to $5 \times 5 = 25$ per channel? **No!** The gaps are empty zeros, not learnable weights:

$$\text{Parameters} = K^2 \cdot C_{\text{in}} \cdot C_{\text{out}} = 3^2 \times 16 \times 32 = 9 \times 512 = 4,608$$

- **Example B (Transposed Convolution Doubling Resolution — `CV03`):** We want to upsample a 16×16 feature map to 32×32 ($O = 2I$) using `nn.ConvTranspose2d` with $D = 1, P_{\text{out}} = 0$.
  - Try $K = 4, S = 2, P = 1$:

$$O = (16 - 1) \times 2 - 2(1) + 4 = 30 - 2 + 4 = 32 = 2 \times 16 \quad \checkmark$$

    Moreover, since $K = 4$ is cleanly divisible by $S = 2$ ($4/2 = 2$), every output pixel receives contributions from exactly $2 \times 2 = 4$ kernel taps—avoiding checkerboard artifacts!

🎯 Exam Takeaway & Common Traps

- **Trap 1 (`CV01`):** Dilation $D > 1$ changes $K_{\text{eff}} = D(K - 1) + 1$ and shrinks the output spatial size O (unless you increase padding to $P = \frac{D(K-1)}{2}$), **but it adds ZERO learnable parameters**.
- **Trap 2 (`CV03`):** Memorize the golden upsampling triplet for `ConvTranspose2d`: $K = 4, S = 2, P = 1$ always gives $O = 2I$ cleanly without checkerboard artifacts.
- **Trap 3:** Always apply the floor function $\lfloor \cdot \rfloor$ **before** adding $+1$ when $S > 1$!

1.2 🌟 [Core — Bread & Butter] Exact Parameter & MACs/FLOPs Formulas Across Convolution Variants (`CV02`, `CV04`)

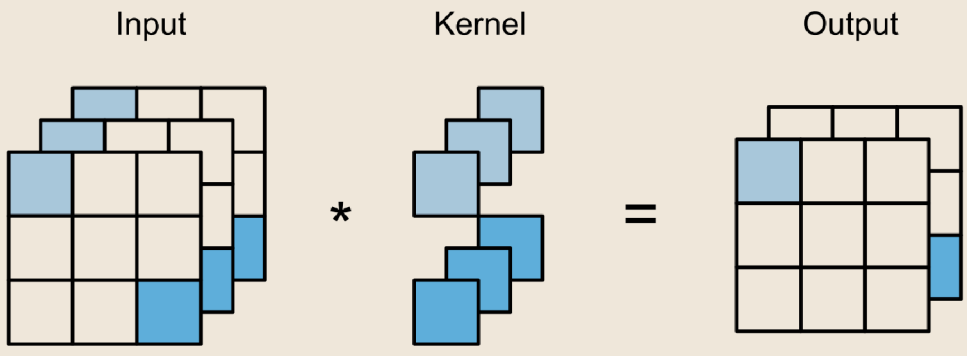
💡 Intuition (Plain English First)

Why is a standard 2D convolution so computationally expensive on mobile and edge chips (like Qualcomm Snapdragon NPUs)? Because a standard $K \times K$ convolution tries to do **two completely different jobs at the exact same time**:

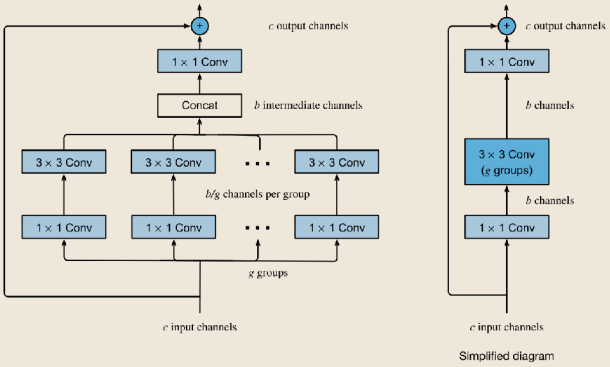
1. **Spatial filtering:** Looking at a $K \times K$ local neighborhood of pixels within each channel.
2. **Cross-channel mixing:** Combining all C_{in} input channels together to produce each of the C_{out} output channels.

Efficient edge architectures decouple or restrict this joint operation in four smart ways:

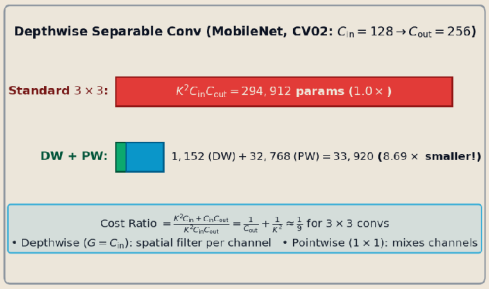
- 1×1 **Pointwise Convolution (`Network-in-Network`):** Shrinks the spatial window to a single pixel (1×1). It does **zero spatial filtering** across neighboring pixels; instead, at every (h, w) pixel location, it takes the vector of C_{in} channels and multiplies it by an $n \times m$ ($n = C_{\text{out}}$ rows, $m = C_{\text{in}}$ columns) weight matrix W. It is mathematically identical to running a **Fully Connected (Linear/MLP) layer independently on every pixel**!
- **Grouped Convolution (`groups = G`):** Splits the C_{in} input channels into G separate, non-talking groups of $\frac{C_{\text{in}}}{G}$ channels each, and assigns $\frac{C_{\text{out}}}{G}$ filters to each group. Because each filter only looks at $\frac{1}{G}$ of the input channels, both parameters and compute drop by an exact factor of G!
- **Depthwise Convolution (`groups = C_in`):** Pushes grouped convolution to the extreme by setting $G = C_{\text{in}} = C_{\text{out}}$. Every single input channel gets its own dedicated $K \times K$ spatial filter, with **zero mixing between different channels**.
- **Depthwise Separable Convolution (`DW + PW`, used in MobileNet/Xception):** Splits a standard convolution into two lightweight steps:
  1. A $K \times K$ **Depthwise Conv** to filter spatial patterns inside each channel independently.
  2. A 1×1 **Pointwise Conv** to mix information across all channels. For a 3×3 filter, this two-step factorization slashes parameters and MACs by $\sim 8\times$ **to** $9\times$ with almost no accuracy loss!



1x1 Pointwise Convolution as a Per-Pixel Linear Layer Across Channels



Grouped Convolution Split-Transform-Merge Topology



Depthwise Separable Convolution (DW + PW) Parameter Reduction

📐 The Math & Every Symbol Explained

Let the input tensor have shape $(C_{\text{in}}, H_{\text{in}}, W_{\text{in}})$ and the output tensor have shape $(C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$ using a $K \times K$ spatial kernel:

- C_{in}: Number of input channels (depth of input feature map).
- C_{out}: Number of output channels (number of filters).
- $H_{\text{out}}, W_{\text{out}}$: Output feature map height and width (forming an $H_{\text{out}} \times W_{\text{out}}$ grid of H_{out} rows and W_{out} columns, containing $H_{\text{out}} W_{\text{out}}$ total output spatial locations).
- K: Spatial kernel size ($K \times K$ weights per channel slice).
- G: Number of channel groups in Grouped Convolution (must divide both C_{in} and C_{out} evenly).
- MACs: Multiply-Accumulate operations. Producing one output value from a dot product of length M requires M multiplications and M additions $= M$ MACs $= 2M$ FLOPs.

Convolution Variant	Learnable Parameters (`bias=False`)	Bias Parameters (`bias=True`)	Total MACs (Multiply-Accumulates)	Cost Ratio vs. Standard Conv
Standard Conv2d	$K^2 C_{\text{in}} C_{\text{out}}$	$+C_{\text{out}}$	$H_{\text{out}} W_{\text{out}} K^2 C_{\text{in}} C_{\text{out}}$	$1.0\times$
1×1 **Pointwise Conv**	$C_{\text{in}} C_{\text{out}}$	$+C_{\text{out}}$	$H_{\text{out}} W_{\text{out}} C_{\text{in}} C_{\text{out}}$	$\frac{1}{K^2}$
Grouped Conv (G **groups,** `CV04` **)**	$\frac{1}{G} K^2 C_{\text{in}} C_{\text{out}}$	$+C_{\text{out}}$ *(NOT* $\frac{C_{\text{out}}}{G}$ *!)*	$\frac{1}{G} H_{\text{out}} W_{\text{out}} K^2 C_{\text{in}} C_{\text{out}}$	$\frac{1}{G}$
Depthwise Conv ($G = C_{\text{in}} = C_{\text{out}}$)	$K^2 C_{\text{in}}$	$+C_{\text{in}}$	$H_{\text{out}} W_{\text{out}} K^2 C_{\text{in}}$	$\frac{1}{C_{\text{out}}}$

Convolution Variant	Learnable Parameters (`bias=False`)	Bias Parameters (`bias=True`)	Total MACs (Multiply-Accumulates)	Cost Ratio vs. Standard Conv
Depthwise Separable (`DW + PW` , `CV02`)	$K^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}$	$+C_{\text{in}} + C_{\text{out}}$	$H_{\text{out}} W_{\text{out}} (K^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}})$	$\frac{1}{C_{\text{out}}} + \frac{1}{K^2}$
Spatially Separable ($K \times 1$ then $1 \times K$)	$2KC_{\text{in}} C_{\text{out}}$ *(if* $mid = C_{\text{in}}$ *)*	$+C_{\text{in}} + C_{\text{out}}$	$2K H_{\text{out}} W_{\text{out}} C_{\text{in}} C_{\text{out}}$	$\frac{2}{K}$ *(Rank-1 filters only)*

Explicitly, the **Depthwise Separable Reduction Ratio** relative to a Standard $K \times K$ Conv is:

$$\frac{\text{Cost(DW + PW)}}{\text{Cost(Standard)}} = \frac{K^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}}{K^2 C_{\text{in}} C_{\text{out}}} = \frac{1}{C_{\text{out}}} + \frac{1}{K^2}$$

🔢 Step-by-Step Worked Toy Example

- **Example A (Standard Conv vs. Depthwise Separable Conv — `CV02`):** Let $C_{\text{in}} = 64$, $C_{\text{out}} = 128$, kernel size $K = 3$, output spatial grid $H_{\text{out}} \times W_{\text{out}} = 32 \times 32$ $(1,024$ pixels), and `bias=False` .
  1. **Standard 3×3 Conv:**

     $$\text{Params}_{\text{std}} = 3^2 \times 64 \times 128 = 9 \times 8,192 = 73,728$$

     $$\text{MACs}_{\text{std}} = 32 \times 32 \times 73,728 = 75,497,472 \text{ MACs}$$

  2. **Depthwise Separable Conv (3×3 DW followed by 1×1 PW):**
     - **Step 2a (Depthwise 3×3 stage):** Each of the 64 input channels gets one 3×3 filter:

       $$\text{Params}_{\text{DW}} = 3^2 \times 64 = 576$$

     - **Step 2b (Pointwise 1×1 stage):** Mixes 64 channels into 128 channels via an $n \times m$ (128×64) weight matrix:

       $$\text{Params}_{\text{PW}} = 1^2 \times 64 \times 128 = 8,192$$

     - **Step 2c (Total DW + PW Parameters & MACs):**

       $$\text{Params}_{\text{DW+PW}} = 576 + 8,192 = 8,768$$

       $$\text{MACs}_{\text{DW+PW}} = 32 \times 32 \times 8,768 = 8,978,432 \text{ MACs}$$

  3. **Reduction Factor Check:**

     $$\frac{73,728}{8,768} \approx 8.41\times \text{ fewer parameters and MACs!} \quad \left(\text{Formula check: } \frac{1}{128} + \frac{1}{9} = \frac{137}{1152} = \frac{1}{8.409}\right)$$

- **Example B (Grouped Convolution Bias Trap — `CV04`):** Suppose a `Conv2d` layer has $C_{\text{in}} = 64$, $C_{\text{out}} = 128$, $K = 3$, `groups = 4` , and `bias = True` . How many total learnable parameters does it have?
  1. **Kernel Weights:** Divided by $G = 4$:

$$\text{Weights} = \frac{3^2 \times 64 \times 128}{4} = \frac{73,728}{4} = 18,432$$

2. **Bias Parameters:** Every one of the $C_{\text{out}} = 128$ output channels still gets its own scalar bias term (biases are **never** divided by G!):

$$\text{Biases} = C_{\text{out}} = 128$$

3. **Total Parameters:** $18,432 + 128 = \mathbf{18,560}$ (a common distractor is $18,432 + 128/4 = 18,464$—don't fall for it!).

🎯 Exam Takeaway & Common Traps

- **Trap 1 (`CV04` Grouped Conv Bias Rule):** `groups = G` divides the **weight** count by G, but if `bias=True`, the bias vector always has C_{out} **parameters** (not C_{out}/G).
- **Trap 2 (`CV02` DW-Sep Bias Rule):** If both the Depthwise ($K \times K$) and Pointwise (1×1) layers have `bias=True`, total biases equal $C_{\text{in}} + C_{\text{out}}$ (in practice, BatchNorm follows each conv, so `bias=False` is used).
- **Trap 3 (MACs vs. FLOPs):** 1 MAC = 2 FLOPs. If an exam question asks for **FLOPs** instead of **MACs**, multiply the MAC count by 2!
- **Trap 4 (Spatially Separable Limitation):** Factoring a $K \times K$ matrix (an $n \times m$ grid with $n = K$ rows and $m = K$ columns) into a column vector ($K \times 1$) times a row vector ($1 \times K$) only works without approximation if the 2D filter matrix has **Rank** $= 1$ (like a Sobel or Gaussian filter). Depthwise Separable Conv splits across the **channel** axis instead, which works for arbitrary 2D spatial filters!

1.3 🌟 [Core — Bread & Butter] Landmark CNN & Edge Architectures: VGG, ResNet, MobileNetV2, GAP & SENet (`CV05`, `CV06`, `CV07`, `CV18`, `CV19`)

💡 Intuition (Plain English First)

How did CNN backbones evolve from heavy desktop models into ultra-fast mobile vision engines? Each landmark architecture introduced one foundational engineering trick:

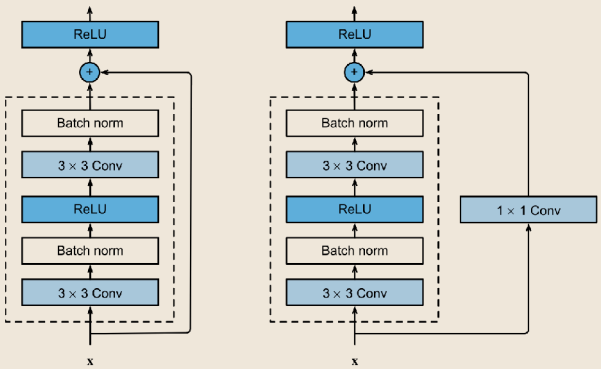
1. **VGG (3×3 Stacking Principle, `CV05`):** Early CNNs (like AlexNet) used huge 7×7 or 11×11 filters in single layers. VGG showed that stacking **three** 3×3 **convolutions** back-to-back covers the **exact same** 7×7 **receptive field** as a single 7×7 layer, while using 45% **fewer parameters** and inserting **three ReLU non-linearities** instead of one!
2. **ResNet (Residual Connections & Bottleneck Blocks, `CV06`, `CV07`):**
   - *Why did plain 56-layer networks perform worse than 20-layer networks even on the **training set**?* Not because of overfitting! It is an **optimization degradation problem**: multiplying Jacobians across 50+ layers causes gradients to vanish or become ill-conditioned, making it nearly impossible for plain layers to even learn an identity mapping $y = x$. By wiring an **identity skip connection** $y = \mathcal{F}(x) + x$, if a layer isn't needed, the network can easily push its weights toward zero ($\mathcal{F}(x) \to 0$) and preserve $y = x$, allowing gradients to flow unimpeded through the $+$ highway!

- *Why the ResNet-50 Bottleneck ($1 \times 1 \to 3 \times 3 \to 1 \times 1$)?* Running a 3×3 conv directly on $C = 256$ channels is expensive. So ResNet-50 first uses a 1×1 **conv to squeeze** channels by $4\times$ ($256 \to 64$), runs the 3×3 **spatial conv in the narrow** 64**-channel space**, and uses a second 1×1 **conv to expand** back ($64 \to 256$).

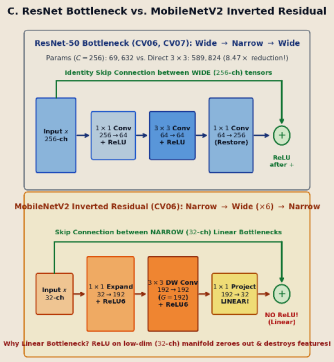
3. **MobileNetV2 (Inverted Residuals & Linear Bottlenecks, `CV06`):**
   - MobileNetV2 flips ResNet's hourglass inside-out! Instead of Wide $\to$ Narrow $\to$ Wide, MobileNetV2 connects skip connections between **narrow** bottleneck tensors (e.g., $C = 32$), uses a 1×1 conv to **expand** channels by $t = 6\times$ ($32 \to 192$), runs a cheap 3×3 **Depthwise Conv** in the wide 192-channel space, and projects back to 32 channels with a 1×1 conv.
   - *Crucial Insight — Why is the final* 1×1 *projection* ***LINEAR*** *(NO ReLU)?* ReLU zeroes out all negative numbers ($\max(0, x)$). In a high-dimensional expanded space (192 channels), there is enough redundancy across channels that zeroing negative activations doesn't destroy the underlying feature manifold. But when you compress down into a tight low-dimensional bottleneck (32 channels), every coordinate carries dense information—applying ReLU there irreversibly crushes half the feature space!

4. **Global Average Pooling (GAP) vs. Flatten + FC (`CV18`):**
   - VGG flattened its final $(512, 7, 7)$ feature map into a $25,088$-dimensional vector and connected it to a 4096 or 1000-unit Fully Connected layer—creating tens of millions of parameters that overfit easily and lock the network to a fixed 224×224 input size.
   - **GAP** simply takes the spatial average of each $H \times W$ (7×7) feature map, collapsing $(C, H, W) \to (C, 1, 1)$ with **zero learnable parameters**!

5. **Squeeze-and-Excitation (SENet Channel Attention, `CV19`):**
   - Standard convolutions weight all feature channels equally when summing them. An **SE Block** asks: "*For this specific image, which channels are most important?*" It **Squeezes** each channel's $H \times W$ spatial map into a single number via GAP, passes that C-dimensional vector through a tiny 2-layer MLP (**Excitation**) with a Sigmoid gate to produce per-channel importance scores $s_c \in (0, 1)$, and **Scales** each channel by s_c.



Plain Convolution Block vs. ResNet Identity Skip Block



ResNet-50 Bottleneck vs. MobileNetV2 Inverted Residual

📐 The Math & Every Symbol Explained

1. **VGG Stacked 3×3 vs. Single $K_{\text{large}} \times K_{\text{large}}$ Layer (`CV05`):**
   - Stacking k layers of 3×3 stride-1 convolutions yields an effective Receptive Field of:

     $$RF = 2k + 1$$

   - For $k = 3$ (matching one 7×7 layer across C channels, `bias=False`):

     $$\text{Params}_{3\times(3\times3)} = 3 \times (3^2C^2) = 27C^2 \quad \text{vs.} \quad \text{Params}_{1\times(7\times7)} = 7^2C^2 = 49C^2$$

2. **ResNet Basic Block vs. Bottleneck Block (`CV06`, `CV07`):**
   - **Residual Mapping:** $y = \mathcal{F}(x, \{W_i\}) + x$ (or $y = \mathcal{F}(x) + W_s x$ using a 1×1 stride-S projection matrix W_s when spatial size or channel count changes).
   - **Basic Block (ResNet-18/34, $C \to C \to C$):** Two 3×3 convs $\implies 2 \times 9C^2 = 18C^2$ parameters.
   - **Bottleneck Block (ResNet-50/101/152, $C \to \frac{C}{4} \to \frac{C}{4} \to C$):**

     $$\text{Params}_{\text{Bottleneck}} = \underbrace{\left(1^2 \cdot C \cdot \frac{C}{4}\right)}_{1\times1 \text{ Reduce}} + \underbrace{\left(3^2 \cdot \frac{C}{4} \cdot \frac{C}{4}\right)}_{3\times3 \text{ Conv}} + \underbrace{\left(1^2 \cdot \frac{C}{4} \cdot C\right)}_{1\times1 \text{ Expand}} = \frac{1}{4}C^2 + \frac{9}{16}C^2 + \frac{1}{4}C^2 = \frac{17}{16}C^2 = 1.06$$

3. **MobileNetV2 Inverted Residual Block (Expansion Ratio t, `CV06`):** For input/output bottleneck channel width C and expansion factor t (typically $t = 6$):

   $$C \xrightarrow[\text{ReLU6}]{1\times1 \text{ Expand}} tC \xrightarrow[\text{ReLU6}]{3\times3 \text{ Depthwise}} tC \xrightarrow[\text{LINEAR (No ReLU!)}]{1\times1 \text{ Project}} C$$

   $$\text{Params}_{\text{MBConv}} = (C \cdot tC) + (3^2 \cdot tC) + (tC \cdot C) = 2tC^2 + 9tC$$

4. **Global Average Pooling (GAP) vs. Flatten + FC (`CV18`):** For a final feature map of shape (C, H, W) feeding a classifier with K_{cls} classes (`bias=False`):

   $$\text{Params}_{\text{Flatten+FC}} = (C \cdot H \cdot W) \cdot K_{\text{cls}} \quad \text{vs.} \quad \text{Params}_{\text{GAP+FC}} = C \cdot K_{\text{cls}} \quad (\text{Reduction factor} = H \times W)$$

5. **Squeeze-and-Excitation (SENet) Block (`CV19`):** Given feature map $X \in \mathbb{R}^{C\times H\times W}$ and bottleneck reduction ratio r (typically $r = 16$):

- **Squeeze (GAP):** $z_c = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} X_c(i,j) \in \mathbb{R}^C$.
- **Excitation (2-Layer Bottleneck MLP):**

$$s = \sigma(W_2 \, \mathrm{ReLU}(W_1 z)) \in (0,1)^C$$

  where $W_1 \in \mathbb{R}^{(C/r) \times C}$ is an $n \times m$ matrix with $n = C/r$ rows and $m = C$ columns, $W_2 \in \mathbb{R}^{C \times (C/r)}$ is an $n \times m$ matrix with $n = C$ rows and $m = C/r$ columns, and $\sigma(\cdot)$ is the element-wise Sigmoid function.
- **SE Parameter Count (`bias=False`):**

$$\mathrm{Params}_{\mathrm{SE}} = \left(\frac{C}{r} \cdot C\right) + \left(C \cdot \frac{C}{r}\right) = \frac{2C^2}{r}$$

🔢 Step-by-Step Worked Toy Example

- **Example A (VGG 3×3 Stacking vs. 7×7 Conv — `CV05`):** Let $C_{\mathrm{in}} = C_{\mathrm{out}} = 64$ (`bias=False`).
  - **One 7×7 layer:** $7^2 \times 64 \times 64 = 49 \times 4,096 = \mathbf{200,704}$ parameters.
  - **Three 3×3 layers:** $3 \times (3^2 \times 64 \times 64) = 27 \times 4,096 = \mathbf{110,592}$ parameters.
  - **Savings:** $\frac{200,704 - 110,592}{200,704} = \frac{22}{49} \approx \mathbf{44.9\%}$ fewer parameters for the exact same 7×7 receptive field!
- **Example B (ResNet Bottleneck vs. Standard 3×3 Conv — `CV07`):** Suppose a residual stage operates at $C = 256$ channels (`bias=False`).
  1. **Direct 3×3 Conv ($256 \to 256$):**

$$\mathrm{Params}_{\mathrm{direct}} = 3^2 \times 256 \times 256 = 9 \times 65,536 = \mathbf{589,824}$$

  2. **ResNet Bottleneck ($256 \xrightarrow{1\times1} 64 \xrightarrow{3\times3} 64 \xrightarrow{1\times1} 256$):**
     - 1×1 Reduce ($256 \to 64$): $1^2 \times 256 \times 64 = 16,384$
     - 3×3 Spatial ($64 \to 64$): $3^2 \times 64 \times 64 = 36,864$
     - 1×1 Expand ($64 \to 256$): $1^2 \times 64 \times 256 = 16,384$
     - **Total Bottleneck Params:** $16,384 + 36,864 + 16,384 = \mathbf{69,632}$ (which matches $\frac{17}{16} \times 256^2 = 69,632$).
  3. **Reduction Ratio:** $\frac{589,824}{69,632} = \frac{9}{17/16} = \frac{144}{17} \approx \mathbf{8.47}\times$ fewer parameters!
- **Example C (GAP vs. Flatten+FC & SENet Parameter Audit — `CV18`, `CV19`):**
  1. **GAP vs. Flatten (`CV18`):** Given a $(C, H, W) = (512, 7, 7)$ feature map and $K_{\mathrm{cls}} = 1000$ classes (`bias=False`):
     - **Flatten + FC:** $(512 \times 7 \times 7) \times 1000 = 25,088 \times 1000 = \mathbf{25,088,000}$ parameters.
     - **GAP + FC:** GAP produces $(512, 1, 1)$ with 0 parameters, followed by $512 \times 1000 = \mathbf{512,000}$ parameters—an exact $7 \times 7 = \mathbf{49}\times$ reduction!
  2. **SENet Block (`CV19`):** Given $C = 256$ channels and reduction ratio $r = 16$ (`bias=False`):
     - Bottleneck dimension: $\frac{C}{r} = \frac{256}{16} = 16$.
     - Total SE parameters: $\frac{2 \times 256^2}{16} = 2 \times 256 \times 16 = \mathbf{8,192}$ parameters.

🎯 Exam Takeaway & Common Traps

- **Trap 1 (`CV06` ResNet vs. MobileNetV2):** ResNet Bottleneck is **Wide** → **Narrow** → **Wide** (skip connection on wide channels); MobileNetV2 Inverted Residual is **Narrow** → **Wide** → **Narrow** (skip connection on narrow channels, and **no ReLU** on the final 1×1 projection).
- **Trap 2 (`CV06` Why ResNet Works):** Skip connections solve **training optimization degradation** (vanishing gradients through deep layers), **not** overfitting!
- **Trap 3 (`CV18` GAP Benefits):** GAP adds **zero** learnable parameters, reduces classifier head parameters by $H \times W$, and allows the network to accept **any input image resolution** at test time.

▼ 📌 **Optional / Stage-2 Interview Bonus (Skip for Timed MCQ): ConvNeXt Modernization Details**

- **How ConvNeXt Modernizes a Standard ResNet-50 to Match Swin Transformer Accuracy:**
  1. **Macro Design:** Replaces the 7×7 stride-2 + MaxPool stem with a ViT-style 4×4 **non-overlapping stride-4 "patchify" stem**, and changes stage block ratios from $(3, 4, 6, 3)$ to $(3, 3, 9, 3)$.
  2. **Depthwise Separable + Large 7×7 Kernel:** Adopts ResNeXt/MobileNet depthwise convolutions and expands the spatial kernel from 3×3 to 7×7 (matching Swin's 7×7 window size).
  3. **Inverted Bottleneck ($C \to 4C \to C$):** Moves the 7×7 depthwise conv to the front of the block (`7x7 DW -> 1x1 Expand (4C) -> GELU -> 1x1 Project (C)`), keeping the heavy 7×7 conv at the narrower channel width C!
  4. **Micro-Design (Transformer Style):** Replaces ReLU with **GELU** (using only **one** activation per block instead of three) and replaces BatchNorm with **LayerNorm** (using only one norm per block).

Day 7: Receptive Field Recursion, Object Detection & Segmentation

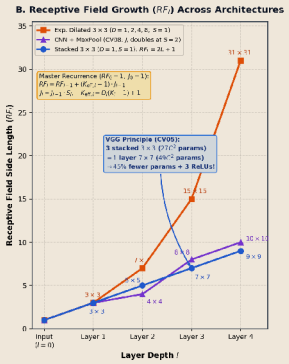
↑ Mục Lục

2.1 🌟 [Core — Bread & Butter] Iterative Receptive Field Calculation (`CV01`, `CV08`)

💡 Intuition (Plain English First)

Pick a single pixel deep inside layer L of a CNN and ask: "*How big of a square patch on the original raw input image could possibly affect this one neuron's value?*" That square footprint is the **Receptive Field (RF_L)**.

- At the raw input image (Layer 0), each pixel only sees itself, so $RF_0 = 1$ and the distance ("jump" J_0) between two adjacent pixels is 1 pixel.
- When you apply a 3×3 convolution at Layer 1, each output pixel looks at 3 adjacent input pixels ($1 + 2 = 3$).
- What happens if Layer 2 is a 2×2 MaxPool with stride $S_2 = 2$? Not only does each pooled pixel merge 2×2 neighbors from Layer 1, it also **doubles the step size ("jump" $J_2 = 2$)** between adjacent features! Because every pixel in Layer 2 now sits 2 raw input pixels apart, any 3×3 convolution placed *after* that pooling layer will reach across twice as many input pixels!



Receptive Field & Cumulative Jump Growth Across Layers

📐 The Math & Every Symbol Explained

To compute the input Receptive Field after L sequential layers (Convolution or Pooling):

- $l \in \{1, 2, \ldots, L\}$: Layer index ($l = 0$ is the raw input image).
- K_l: Physical kernel size of layer l.
- D_l: Dilation rate of layer l ($D_l = 1$ for standard conv or pooling).
- $K_{\text{eff},l} = D_l(K_l - 1) + 1$: Effective kernel size of layer l.
- S_l: Stride of layer l.
- J_l: Cumulative stride (the "jump" or spacing on the input image between two adjacent pixels in layer l's feature map).
- RF_l: Receptive field width/height (in raw input image pixels) of a single neuron at the output of layer l.

1. **Initialization at the Input Image ($l = 0$):**

$$RF_0 = 1, \qquad J_0 = 1$$

2. **Layer-by-Layer Forward Recursion ($l = 1, 2, \ldots, L$):**

$$RF_l = RF_{l-1} + (K_{\text{eff},l} - 1) \times J_{l-1}$$

$$J_l = J_{l-1} \times S_l = \prod_{i=1}^{l} S_i$$

*(Notice that RF_l uses the **previous** layer's jump J_{l-1}, and then J_l updates for the next layer!)*

🔢 Step-by-Step Worked Toy Example

Let's trace the exact Receptive Field RF_l and Jump J_l through a 4-layer network (`CV08` , `CV01`):

- **Layer 1:** 3×3 Conv, stride $S_1 = 1$, padding $P_1 = 1$, dilation $D_1 = 1$.
- **Layer 2:** 2×2 MaxPool, stride $S_2 = 2$, padding $P_2 = 0$.
- **Layer 3:** 3×3 Dilated Conv, stride $S_3 = 1$, padding $P_3 = 2$, dilation $D_3 = 2$.
- **Layer 4:** 1×1 Pointwise Conv, stride $S_4 = 1$, padding $P_4 = 0$, dilation $D_4 = 1$.

Let's compute step by step starting from $RF_0 = 1, J_0 = 1$:

1. **After Layer 1 ($K_1 = 3, D_1 = 1 \implies K_{\text{eff},1} = 3, S_1 = 1$):**

$$RF_1 = RF_0 + (3-1) \times J_0 = 1 + 2 \times 1 = \mathbf{3}, \qquad J_1 = 1 \times 1 = \mathbf{1}$$

2. **After Layer 2 ($K_2 = 2, D_2 = 1 \implies K_{\text{eff},2} = 2, S_2 = 2$):**

$$RF_2 = RF_1 + (2-1) \times J_1 = 3 + 1 \times 1 = \mathbf{4}, \qquad J_2 = 1 \times 2 = \mathbf{2}$$

(The 2×2 MaxPool only added 1 pixel to RF_2, but it doubled the jump J_2 to 2!)

3. **After Layer 3 ($K_3 = 3, D_3 = 2 \implies K_{\text{eff},3} = 2(3-1)+1 = 5, S_3 = 1$):**

$$RF_3 = RF_2 + (5-1) \times J_2 = 4 + 4 \times 2 = \mathbf{12}, \qquad J_3 = 2 \times 1 = \mathbf{2}$$

(Because $J_2 = 2$ and dilation made $K_{\text{eff},3} = 5$, Layer 3 added $4 \times 2 = 8$ pixels to the receptive field!)

4. **After Layer 4 ($K_4 = 1, D_4 = 1 \implies K_{\text{eff},4} = 1, S_4 = 1$):**

$$RF_4 = RF_3 + (1-1) \times J_3 = 12 + 0 \times 2 = \mathbf{12}, \qquad J_4 = 2 \times 1 = \mathbf{2}$$

(A 1×1 stride-1 conv never changes RF or J!)

🎯 Exam Takeaway & Common Traps

- **Trap 1: Padding P_l NEVER affects the Receptive Field RF_l!** Padding only shifts border coordinates and changes the output spatial dimensions $H_{\text{out}} \times W_{\text{out}}$. Ignore P_l completely when computing RF_l.
- **Trap 2: 1×1 Convolutions ($S = 1$) NEVER change RF_l** because $K_{\text{eff}} - 1 = 1 - 1 = 0$.
- **Trap 3 (`CV08`):** Don't multiply RF_{l-1} by 2 when hitting a 2×2 stride-2 MaxPool! A 2×2 MaxPool adds $(2-1) \times J_{l-1}$ to RF_{l-1} and doubles J_l for the *subsequent* layers.

2.2 🌟 [Core — Bread & Butter] Object Detection Part 1: IoU, GIoU, NMS & Feature Pyramid Networks (`CV09`, `CV10`)

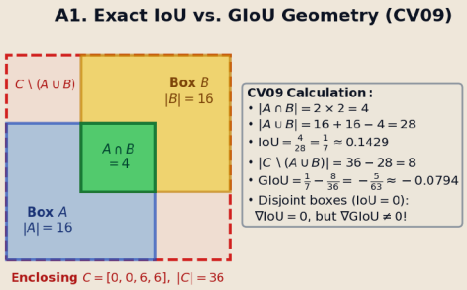
💡 Intuition (Plain English First)

In object detection, we need to draw a tight rectangular bounding box around every object and classify it:

- **Intersection over Union (IoU):** Measures how well a predicted box A lines up with a ground-truth box B by dividing their **overlapping area** by their **total combined (union) area**. If they match perfectly, IoU $= 1$; if they don't touch at all, IoU $= 0$.
- **Why IoU Fails as a Loss Function (and how GIoU Fixes It, `CV09`):** Suppose your predicted box A and ground-truth box B do not overlap yet ($|A \cap B| = 0$). Then IoU $= 0$ whether the boxes are 1 pixel apart or $1,000$ pixels apart! Because the loss $\mathcal{L}_{\text{IoU}} = 1 - \text{IoU} = 1$ is completely flat everywhere outside overlap, its **gradient is zero** ($\nabla \text{IoU} = 0$) and the network has no idea which direction to move the box! **Generalized IoU (GIoU)** fixes this by drawing the smallest enclosing box C around both A and B, and subtracting the fraction of "empty wasted space" inside C. Even when A and B don't touch, moving A closer to B shrinks the enclosing box C, giving a smooth, non-zero gradient!
- **Non-Maximum Suppression (NMS, `CV10`):** A detector often fires 5 or 10 slightly shifted boxes on the exact same car. **Greedy NMS** cleans up these duplicates **per class**: it grabs the highest-scoring box B^*, keeps it, and

deletes every other box of the same class whose overlap with B^* exceeds a threshold (IoU $> \theta_{\text{NMS}}$). However, in a crowded street where two real pedestrians stand right in front of each other (IoU $> \theta_{\text{NMS}}$), hard NMS accidentally deletes the second pedestrian! **Soft-NMS** fixes this by smoothly decaying the overlapping box's confidence score instead of immediately zeroing it out.

- **Feature Pyramid Network (FPN):** Early CNN layers have sharp spatial resolution (great for tiny objects) but weak semantic meaning; deep CNN layers have rich semantic meaning (great for recognizing object categories) but coarse spatial resolution. **FPN** builds a top-down bridge that upsamples deep semantic maps and adds them to shallow high-resolution maps (via 1×1 lateral channel projections), giving the detector rich semantics at **every** spatial scale!



Exact IoU vs. GIoU Enclosing Box Geometry

📐 The Math & Every Symbol Explained

Let two axis-aligned 2D bounding boxes be defined by their top-left and bottom-right coordinates $A = [x_{1A}, y_{1A}, x_{2A}, y_{2A}]$ and $B = [x_{1B}, y_{1B}, x_{2B}, y_{2B}]$:

- $|A| = (x_{2A} - x_{1A})(y_{2A} - y_{1A})$: Area of box A (and similarly $|B|$ for box B).
- $|A \cap B|$: Area of the intersection rectangle:

$$w_{\cap} = \max(0,\, \min(x_{2A}, x_{2B}) - \max(x_{1A}, x_{1B}))\,, \quad h_{\cap} = \max(0,\, \min(y_{2A}, y_{2B}) - \max(y_{1A}, y_{1B}))$$

$$|A \cap B| = w_{\cap} \times h_{\cap}$$

- $|A \cup B| = |A| + |B| - |A \cap B|$: Area of the union of boxes A and B (we subtract $|A \cap B|$ so the overlapping region isn't double-counted!).
- C: The smallest axis-aligned enclosing rectangle that completely covers both A and B:

$$x_{1C} = \min(x_{1A}, x_{1B}), \quad y_{1C} = \min(y_{1A}, y_{1B}), \quad x_{2C} = \max(x_{2A}, x_{2B}), \quad y_{2C} = \max(y_{2A}, y_{2B})$$

$$|C| = (x_{2C} - x_{1C})(y_{2C} - y_{1C})$$

1. **Intersection over Union (**$\text{IoU} \in [0, 1]$**):**

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

2. **Generalized IoU (**$\text{GIoU} \in (-1, 1]$**,** `CV09`**):**

$$\text{GIoU}(A, B) = \text{IoU}(A, B) - \frac{|C \setminus (A \cup B)|}{|C|} = \text{IoU}(A, B) - \frac{|C| - |A \cup B|}{|C|}$$

- **Range limits:** GIoU $= 1$ when $A = B$ perfectly; GIoU $\to -1$ as the distance between non-overlapping boxes approaches infinity ($\frac{|A \cup B|}{|C|} \to 0$).

3. **Greedy NMS vs. Linear Soft-NMS Score Update (`CV10`):** Given highest-scoring box B^* and a remaining candidate box B_i with confidence score s_i:

$$s_i^{\text{Hard-NMS}} = \begin{cases} s_i & \text{if IoU}(B^*, B_i) \le \theta_{\text{NMS}} \\ 0 & \text{if IoU}(B^*, B_i) > \theta_{\text{NMS}} \end{cases}, \qquad s_i^{\text{Soft-NMS}} = \begin{cases} s_i & \text{if IoU}(B^*, B_i) \le \theta_{\text{NMS}} \\ s_i \left(1 - \text{IoU}(B^*, B_i)\right) & \text{if IoU}(B^*, B_i) > \theta_{\text{NMS}} \end{cases}$$

🔢 Step-by-Step Worked Toy Example

- **Example A (Overlapping Boxes IoU & GIoU — `CV09`):** Let Box $A = [0, 0, 4, 4]$ and Box $B = [2, 2, 6, 6]$.
  1. **Individual Areas:** $|A| = (4 - 0)(4 - 0) = 16$, and $|B| = (6 - 2)(6 - 2) = 16$.
  2. **Intersection Area $|A \cap B|$:**
     - Overlap along x: $[\max(0, 2), \min(4, 6)] = [2, 4] \implies w_\cap = 4 - 2 = 2$.
     - Overlap along y: $[\max(0, 2), \min(4, 6)] = [2, 4] \implies h_\cap = 4 - 2 = 2$.
     - $|A \cap B| = 2 \times 2 = \mathbf{4}$.
  3. **Union Area $|A \cup B|$:**

$$|A \cup B| = |A| + |B| - |A \cap B| = 16 + 16 - 4 = \mathbf{28}$$

  4. **Exact IoU:**

$$\text{IoU}(A, B) = \frac{4}{28} = \frac{1}{7} \approx \mathbf{0.1429}$$

  5. **Smallest Enclosing Box C and GIoU:**
     - $C = [\min(0, 2), \min(0, 2), \max(4, 6), \max(4, 6)] = [0, 0, 6, 6] \implies |C| = 6 \times 6 = \mathbf{36}$.
     - Empty area inside C: $|C| - |A \cup B| = 36 - 28 = 8$.
     - $\text{GIoU}(A, B) = \frac{1}{7} - \frac{8}{36} = \frac{1}{7} - \frac{2}{9} = -\frac{5}{63} \approx \mathbf{-0.0794}$.
- **Example B (Disjoint Boxes Showing Why GIoU Beats IoU — `CV09`):** Let $A = [0, 0, 2, 2]$ ($|A| = 4$) and $B = [4, 4, 6, 6]$ ($|B| = 4$).
  - They do not touch: $|A \cap B| = 0 \implies |A \cup B| = 4 + 4 - 0 = 8$, so IoU $= 0$ (zero gradient!).
  - Enclosing box $C = [0, 0, 6, 6] \implies |C| = 36$.
  - GIoU $= 0 - \frac{36 - 8}{36} = -\frac{28}{36} = \mathbf{-0.7778}$ (non-zero gradient that pulls A and B together!).
- **Example C (Greedy NMS vs. Soft-NMS Step-by-Step — `CV10`):** Suppose a `'car'` detector outputs 3 boxes: B_1 ($s_1 = 0.95$), B_2 ($s_2 = 0.80$), and B_3 ($s_3 = 0.70$), with $\text{IoU}(B_1, B_2) = 0.60$, $\text{IoU}(B_1, B_3) = 0.20$, and $\text{IoU}(B_2, B_3) = 0.10$. Let $\theta_{\text{NMS}} = 0.50$.
  - **Round 1:** Select highest-scoring box $B^* = B_1$ (0.95) and move it to the keep list.
  - **Compare remaining boxes against B_1:**

- B_2 has $\text{IoU}(B_1, B_2) = 0.60 > 0.50$: **Hard NMS** deletes B_2 ($s_2 \leftarrow 0$); **Linear Soft-NMS** decays B_2's score to $s_2 \leftarrow 0.80 \times (1 - 0.60) = \mathbf{0.32}$.
- B_3 has $\text{IoU}(B_1, B_3) = 0.20 \leq 0.50$: Kept untouched ($s_3 = 0.70$).

- **Final Survivors (Hard NMS):** $[B_1, B_3]$.

🎯 Exam Takeaway & Common Traps

- **Trap 1 (`CV09`):** Standard IoU is in $[0, 1]$ and has **zero gradient** for non-overlapping boxes; **GIoU** is in $(-1, 1]$ and always provides a usable gradient by penalizing the empty area $|C| - |A \cup B|$ of the enclosing box C.
- **Trap 2 (`CV10`):** Standard NMS runs **independently per class** in $\mathcal{O}(M^2)$ pairwise comparisons for M candidate boxes, and its main failure mode is **suppressing true-positive objects that overlap heavily in crowded scenes** (fixed by Soft-NMS).
- **Trap 3 (FPN Lateral Convs):** FPN uses 1×1 **lateral convolutions** to project different backbone stages (C_3, C_4, C_5 with $512, 1024, 2048$ channels) to a **single uniform channel width ($d = 256$)** so the top-down $2\times$ upsampled features can be combined via **element-wise addition**, followed by a 3×3 anti-aliasing conv.

<details>
<summary>📌 Optional Deep-Dive (Skip for Timed MCQ): DIoU & CIoU Exact Penalty Formulas</summary>

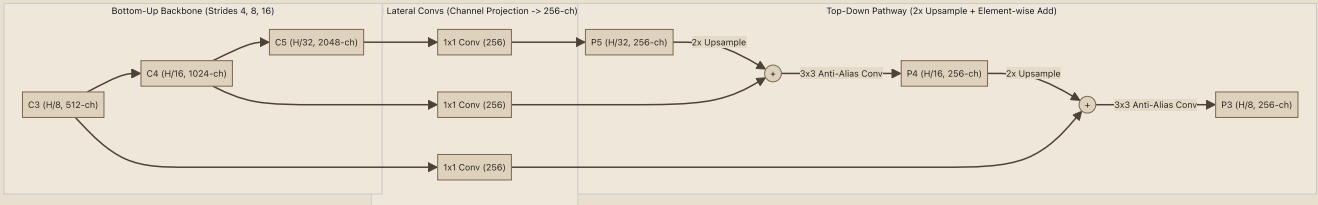
- **Why GIoU Can Still Be Slow:** When one box is completely inside another ($A \subset B$), $|C| = |B| = |A \cup B|$, so the GIoU penalty term $\frac{|C| - |A \cup B|}{|C|} = 0$ vanishes and GIoU degenerates back into standard IoU!
- **DIoU (Distance-IoU):** Directly minimizes the normalized squared Euclidean distance $\rho^2(\mathbf{b}, \mathbf{b}^{gt})$ between the center points of the predicted box $\mathbf{b}$ and ground-truth box $\mathbf{b}^{gt}$, divided by the squared diagonal length c^2 of the smallest enclosing box C:

$$\mathcal{L}_{\text{DIoU}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2}$$

- **CIoU (Complete-IoU):** Adds an aspect-ratio consistency penalty αv where $v = \frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w}{h}\right)^2$:

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v$$

</details>

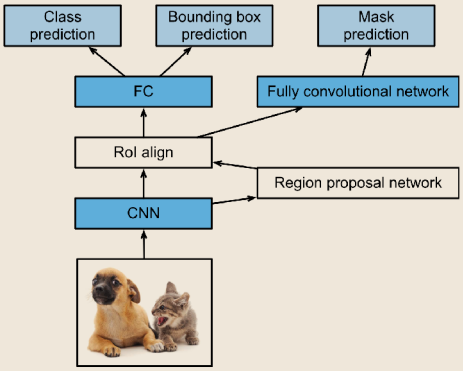


2.3 🌟 [Core — Bread & Butter] Object Detection Part 2: Two-Stage vs. One-Stage, RoI Align (2D Bilinear Interpolation), Focal Loss, DETR & mAP (`CV11`, `CV12`)

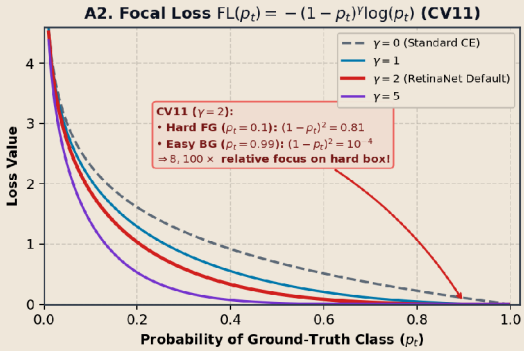
💡 Intuition (Plain English First)

Modern object detectors fall into three architectural families:

1. **Two-Stage Detectors (Faster R-CNN / Mask R-CNN, `CV12`):**
    - *Stage 1:* A **Region Proposal Network (RPN)** scans the feature map and asks a simple class-agnostic question: "*Could there be ANY object inside this region?*"
    - *Stage 2:* For each promising Region of Interest (RoI), we crop out a fixed-size feature grid (e.g., 7×7) and feed it to a classification and bounding-box refinement head.
    - *Why RoI Align Beats RoI Pooling (`CV12`):* Suppose an RPN proposal has width $w = 135$ pixels on the image, and our backbone has stride $S = 16$. On the feature map, the box width is $\frac{135}{16} = 8.4375$ cells. **RoI Pooling** brutally rounds that floating-point coordinate down to an integer ($\lfloor 8.4375 \rfloor = 8$), and then rounds *again* when dividing the 8-cell region into 7×7 bins ($\lfloor 8/7 \rfloor = 1$)! At stride 16, a 0.5-cell rounding error shifts the box by 8 pixels on the original image—destroying pixel-level mask accuracy. **RoI Align** bans all integer rounding (`floor`), keeps exact floating-point coordinates, and uses **2D Bilinear Interpolation** to smoothly sample feature values between the 4 nearest pixel centers!
2. **One-Stage Detectors (YOLO, SSD, RetinaNet, FCOS, `CV11`):**
    - Skip the RPN proposal step and predict class probabilities and boxes directly at every grid cell in a single pass.
    - *The Extreme Class Imbalance Problem & Focal Loss (`CV11`):* A dense one-stage grid evaluates $\sim 100,000$ candidate anchor locations per image, but only ~ 10 contain real foreground objects—the other $99,990$ are easy background sky/road/wall! Even if each easy background box has a tiny cross-entropy loss of 0.01, summing $100,000 \times 0.01 = 1,000$ completely drowns out the few hard foreground objects! **Focal Loss** multiplies standard cross-entropy by $(1 - p_t)^\gamma$: when the model is already 99% sure an anchor is background ($p_t = 0.99$), $(1 - 0.99)^2 = 0.0001$ silences its gradient by $10,000\times$, letting the optimizer focus purely on the hard mistakes!
    - *Anchor-Based vs. Anchor-Free:* Anchor-based models (RetinaNet, YOLOv3) place k predefined reference boxes of fixed scales/aspect ratios at each grid cell and predict offsets $(\Delta x, \Delta y, \Delta w, \Delta h)$. **Anchor-Free** models (FCOS, CenterNet, YOLOv8) drop predefined anchors completely and regress the 4 distances (l, t, r, b) from each feature point directly to the left, top, right, and bottom box edges.
3. **End-to-End Transformer Detectors (DETR, `CV12`):**
    - Why do traditional detectors need hand-designed Anchor Boxes and post-processing NMS? Because they allow *many* neighboring anchors to predict the same object! **DETR** feeds CNN features into a Transformer Encoder-Decoder with $N = 100$ learnable **Object Queries**. During training, it runs the **Hungarian Algorithm** to find the single best **1-to-1 bipartite matching** between the 100 queries and the ground-truth objects (assigning remaining queries to $\varnothing$ "no object"). Because the loss strictly penalizes duplicate predictions during training, DETR outputs exactly one box per object and **needs ZERO Anchor Boxes and ZERO NMS**!



Two-Stage Mask R-CNN Architecture with RPN & RoI Align



Focal Loss Modulating Factor Across Ground-Truth Probabilities

📐 The Math & Every Symbol Explained

1. **2D Bilinear Interpolation (Used Inside RoI Align, `CV12`):** Suppose we want to evaluate a feature map at a continuous sub-pixel coordinate (x, y) that lies inside a unit grid cell whose four surrounding integer grid corners are $(x_0, y_0), (x_1, y_0), (x_0, y_1), (x_1, y_1)$ with known feature values $Q_{00}, Q_{10}, Q_{01}, Q_{11}$. Let the fractional offsets be $\Delta x = x - x_0 \in [0, 1]$ and $\Delta y = y - y_0 \in [0, 1]$:
   - **Step 1 (Interpolate horizontally along top y_0 and bottom y_1):**

$$f(x, y_0) = (1 - \Delta x)\, Q_{00} + \Delta x\, Q_{10}, \qquad f(x, y_1) = (1 - \Delta x)\, Q_{01} + \Delta x\, Q_{11}$$

   - **Step 2 (Interpolate vertically along y):**

$$f(x, y) = (1 - \Delta y)\, f(x, y_0) + \Delta y\, f(x, y_1)$$

$$f(x, y) = (1 - \Delta x)(1 - \Delta y) Q_{00} + \Delta x (1 - \Delta y) Q_{10} + (1 - \Delta x) \Delta y\, Q_{01} + \Delta x\, \Delta y\, Q_{11}$$

2. **Focal Loss (`CV11`):** Let $p_t \in (0, 1]$ be the model's predicted probability for the **ground-truth class** (i.e., $p_t = p$ if $y = 1$, and $p_t = 1 - p$ if $y = 0$):

$$\mathrm{CE}(p_t) = -\log(p_t) \quad \longrightarrow \quad \mathrm{FL}(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t)$$

   - $\alpha_t \in [0, 1]$: Class-balancing weight (typically $\alpha = 0.25$ for foreground).
   - $\gamma \geq 0$: Focusing parameter (default $\gamma = 2$; when $\gamma = 0$, Focal Loss reduces to α-balanced Cross-Entropy).
   - $(1 - p_t)^{\gamma}$: Modulating factor that exponentially suppresses well-classified examples ($p_t \to 1$).

3. **DETR Hungarian Bipartite Matching (`CV12`):** Given N predicted outputs $\hat{y} = \{\hat{y}_i\}_{i=1}^{N}$ and ground-truth set y padded with $\varnothing$ ("no object") to size N, DETR builds an $N \times N$ (N rows, N columns) pairwise matching cost matrix $\mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$ and finds the optimal permutation $\hat{\sigma} \in \mathfrak{S}_N$ via the Hungarian algorithm:

$$\hat{\sigma} = \arg\min_{\sigma \in \mathfrak{S}_N} \sum_{i=1}^{N} \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$$

4. **Mean Average Precision (mAP):**
   - At a given IoU threshold τ_{IoU} (e.g., 0.50), sort all predictions for class c by confidence score descending. Each prediction is a **True Positive (TP)** if IoU $\geq \tau_{\text{IoU}}$ with an unmatched ground-truth box of class c; otherwise (or if it is a **duplicate** detection on an already-matched GT box!) it is a **False Positive (FP)**. Unmatched GT boxes are **False Negatives (FN)**.
   - $\text{Precision} = \frac{\text{TP}}{\text{TP}+\text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP}+\text{FN}}, \quad \text{AP}_c = \int_0^1 P_c(R)\, dR, \quad \text{mAP} = \frac{1}{C}\sum_{c=1}^{C} \text{AP}_c.$
   - **PASCAL VOC `mAP@0.50`:** Evaluates at a single fixed threshold $\tau_{\text{IoU}} = 0.50$.
   - **COCO `mAP@[.50:.05:.95]`:** Averages mAP across **10 IoU thresholds** ($\tau_{\text{IoU}} \in \{0.50, 0.55, 0.60, \ldots, 0.95\}$) to strictly reward tight localization.

🔢 Step-by-Step Worked Toy Example

- **Example A (RoI Align 2D Bilinear Interpolation at a Sub-Pixel Point — `CV12`):** Suppose an RoI Align sampling point lands at continuous coordinate $(x, y) = (0.2, 0.6)$ inside the unit square $[0,1] \times [0,1]$, where the four corner feature values are:
  - Top-left $Q_{00} = f(0,0) = 10$, Top-right $Q_{10} = f(1,0) = 20$
  - Bottom-left $Q_{01} = f(0,1) = 30$, Bottom-right $Q_{11} = f(1,1) = 40$

  1. **Step 1 (Horizontal interpolation at $y_0 = 0$ and $y_1 = 1$ using $\Delta x = 0.2$):**

  $$f(0.2, 0) = (1 - 0.2)(10) + (0.2)(20) = 8 + 4 = \mathbf{12}$$

  $$f(0.2, 1) = (1 - 0.2)(30) + (0.2)(40) = 24 + 8 = \mathbf{32}$$

  2. **Step 2 (Vertical interpolation at $\Delta y = 0.6$ between 12 and 32):**

  $$f(0.2, 0.6) = (1 - 0.6)(12) + (0.6)(32) = 4.8 + 19.2 = \mathbf{24.0}$$

  (Notice how smooth and differentiable this is compared to nearest-neighbor rounding, which would snap $(0.2, 0.6) \to (0, 1)$ and output 30!)

- **Example B (Focal Loss Modulating Factor Comparison — `CV11`):** Let $\gamma = 2$ and $\alpha_t = 1$. Compare the Focal Loss modulating factor $(1 - p_t)^\gamma$ for an **easy background anchor** ($p_t = 0.99$) vs. a **hard foreground anchor** ($p_t = 0.10$):

  1. **Easy Background ($p_t = 0.99$):**

  $$(1 - 0.99)^2 = (0.01)^2 = \mathbf{10^{-4}} = 0.0001 \quad \text{(loss scaled down by } 10,000\times!)$$

  2. **Hard Foreground ($p_t = 0.10$):**

$$(1-0.10)^2 = (0.90)^2 = \mathbf{0.81} \quad \text{(keeps 81\% of its full cross-entropy weight!)}$$

3. **Relative Emphasis Ratio:** $\frac{0.81}{0.0001} = \mathbf{8,100}\times$ more relative weight on the hard example compared to standard cross-entropy!

🎯 Exam Takeaway & Common Traps

- **Trap 1 (`CV12` RoI Align vs. RoI Pooling):** RoI Pooling uses integer rounding (`floor`), causing misalignment; RoI Align uses **exact floating-point coordinates + 2D bilinear interpolation** (with **zero** learnable parameters!).
- **Trap 2 (`CV11` Focal Loss γ):** When $\gamma = 0$, Focal Loss equals standard Cross-Entropy. As γ increases (e.g., $\gamma = 2$), easy examples ($p_t \to 1$) are exponentially down-weighted.
- **Trap 3 (`CV12` DETR):** DETR uses **Hungarian 1-to-1 bipartite matching** + N learnable **Object Queries**, eliminating both **Anchor Boxes** and **NMS**.
- **Trap 4 (mAP Duplicate Detections):** If a model predicts **two** boxes with IoU ≥ 0.50 on the **same** ground-truth object, only the highest-scoring box counts as a **TP**—the second box is penalized as a **False Positive (FP)**!

2.4 🌟 [Core — Bread & Butter] Semantic, Instance & Panoptic Segmentation (`CV13`, `CV14`)

💡 Intuition (Plain English First)

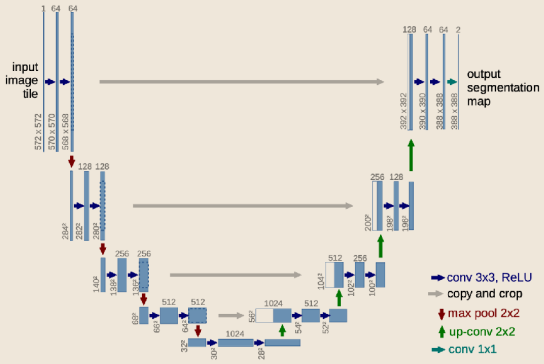
What is the difference between the three core pixel-level segmentation tasks (`CV13`)?

- **Semantic Segmentation:** Colors every pixel by its **category** (`'road'`, `'sky'`, `'person'`, `'car'`), but does **not** separate individual instances of the same class. If 5 people stand next to each other, all their pixels just say `'person'`—you can't tell where Person #1 ends and Person #2 begins!
- **Instance Segmentation (Mask R-CNN):** Detects only countable foreground objects ("**things**" like `'person'` or `'car'`) and gives each individual object its own separate binary mask (`'Person #1'`, `'Person #2'`), while ignoring uncountable background regions ("**stuff**" like `'sky'` or `'grass'`).
- **Panoptic Segmentation:** Unifies both! Every single pixel in the image gets a semantic category label (covering both "stuff" and "things") **AND** an instance ID if it belongs to a countable object.

How do segmentation networks preserve sharp pixel boundaries while understanding global context, and how are they trained (`CV13`, `CV14`)?

- **FCN vs. U-Net vs. DeepLabv3+ (`CV13`):**
  - **FCN (Fully Convolutional Network):** Fuses upsampled decoder features with encoder skip features via **element-wise addition** (`+`), keeping channel count unchanged.
  - **U-Net:** Fuses upsampled decoder features with encoder skip features via **channel-wise concatenation** (`torch.cat` **along** `dim=1`), which **doubles the channel count** entering the decoder convolution so the network can learn how to blend fine edge details with deep semantics!
  - **DeepLabv3+ (ASPP — Atrous Spatial Pyramid Pooling):** Runs multiple **dilated convolutions in parallel** at different dilation rates (e.g., $D = 6, 12, 18$) plus global average pooling to capture small, medium, and large spatial context simultaneously without downsampling.

- **Mask R-CNN's Decoupled Mask Head Trick (`CV13`):** Instead of forcing all K classes to compete at every pixel via a K-way Softmax, Mask R-CNN outputs K separate binary masks (one per class) and applies **per-pixel Sigmoid + Binary Cross-Entropy (BCE) Loss ONLY on the true class k^*'s mask channel**! Once the box head already knows the object is a `'dog'`, the mask head just traces the foreground silhouette without cross-class competition.
- **Dice Loss vs. Pixel-Wise BCE (`CV14`):** Suppose you are segmenting a tiny tumor or thin blood vessel that occupies only 0.5% of the image (99.5% background). A model that lazily predicts "100% background everywhere" gets 99.5% accuracy and a tiny Pixel-Wise BCE loss! **Soft Dice Loss** ignores the massive pool of true negatives (TN) and directly optimizes the **F1 / overlap ratio** between predicted foreground and true foreground!



U-Net Encoder-Decoder Architecture with Skip Concatenations

📐 The Math & Every Symbol Explained

For binary segmentation of a foreground pixel set P (predicted) against ground-truth foreground pixel set G in terms of confusion matrix pixel counts TP (True Positives), FP (False Positives), and FN (False Negatives):

1. **IoU (Jaccard Index) vs. Dice Coefficient (F1 Score, `CV14`):**

$$\text{IoU} = \frac{|P \cap G|}{|P \cup G|} = \frac{|P \cap G|}{|P| + |G| - |P \cap G|} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}}$$

$$\text{Dice} = \frac{2\,|P \cap G|}{|P| + |G|} = \frac{2\,\text{TP}}{2\,\text{TP} + \text{FP} + \text{FN}} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = F_1$$

2. **Exact Algebraic Conversion Between IoU and Dice (`CV14`):**

$$\text{Dice} = \frac{2 \cdot \text{IoU}}{1 + \text{IoU}} \qquad \Longleftrightarrow \qquad \text{IoU} = \frac{\text{Dice}}{2 - \text{Dice}}$$

(Important inequality: For all $\text{IoU} \in (0, 1)$*, we always have* $\text{Dice} > \text{IoU}$*, with equality* $\text{Dice} = \text{IoU}$ *only at* 0 *and* 1*!)*

3. **Differentiable Soft Dice Loss (for continuous probabilities $p_i \in [0, 1]$ and binary labels $g_i \in \{0, 1\}$):**

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2\sum_{i=1}^{M} p_i g_i + \epsilon}{\sum_{i=1}^{M} p_i + \sum_{i=1}^{M} g_i + \epsilon}$$

where $M = H \times W$ is total pixels in the $H \times W$ (H rows, W columns) mask and $\epsilon \approx 10^{-6}$ prevents division by zero when both P and G are empty.

🔢 Step-by-Step Worked Toy Example

- **Example A (Converting Between IoU and Dice — `CV14`):** Suppose a segmentation model achieves IoU $= 0.60$ on the validation set. What is its exact Dice (F1) score?

  $$\text{Dice} = \frac{2 \times 0.60}{1 + 0.60} = \frac{1.20}{1.60} = \mathbf{0.75}$$

  Conversely, if a paper reports Dice $= 0.80$, what is the exact IoU?

  $$\text{IoU} = \frac{0.80}{2 - 0.80} = \frac{0.80}{1.20} = \frac{2}{3} \approx \mathbf{0.6667}$$

- **Example B (Computing TP, FP, FN, IoU & Dice on a 4×4 Mask Grid — `CV14`):** Suppose an image has $4 \times 4 = 16$ pixels. The ground-truth mask G has 5 foreground pixels ($|G| = 5$), the model predicts 4 foreground pixels ($|P| = 4$), and 3 of those predicted pixels overlap with the ground truth ($|P \cap G| = 3$):

  1. **Confusion Matrix Counts:**
     - $\text{TP} = |P \cap G| = \mathbf{3}$
     - $\text{FP} = |P| - \text{TP} = 4 - 3 = \mathbf{1}$
     - $\text{FN} = |G| - \text{TP} = 5 - 3 = \mathbf{2}$
     - $\text{TN} = 16 - (3 + 1 + 2) = \mathbf{10}$ (Notice that neither IoU nor Dice uses TN = 10!)
  2. **IoU & Dice:**

     $$\text{IoU} = \frac{3}{3 + 1 + 2} = \frac{3}{6} = \mathbf{0.50}, \qquad \text{Dice} = \frac{2(3)}{2(3) + 1 + 2} = \frac{6}{9} = \frac{2}{3} \approx \mathbf{0.6667}$$

     (Check conversion formula: $\frac{2(0.50)}{1+0.50} = \frac{1.0}{1.5} = \frac{2}{3}$✓)

🎯 Exam Takeaway & Common Traps

- **Trap 1 (`CV13` U-Net Skip Connections):** If an encoder stage has $C = 128$ channels and the upsampled decoder feature map has 128 channels, U-Net's concatenation (`torch.cat`) produces a **256-channel tensor** entering the decoder's 3×3 convolution (whereas FCN / ResNet addition keeps it at 128).
- **Trap 2 (`CV13` Mask R-CNN Loss):** Mask R-CNN uses **per-pixel Sigmoid + BCE only on the ground-truth class channel**, NOT per-pixel multi-class Softmax!
- **Trap 3 (`CV14` Dice vs. IoU):** Memorize Dice $= \frac{2\,\text{IoU}}{1+\text{IoU}}$. Because the denominator $1 + \text{IoU} < 2$ whenever IoU < 1, **Dice is always strictly greater than IoU** for any partial overlap.

Day 8: Vision Transformers (ViT, Swin), Multimodal (CLIP) & Diffusion

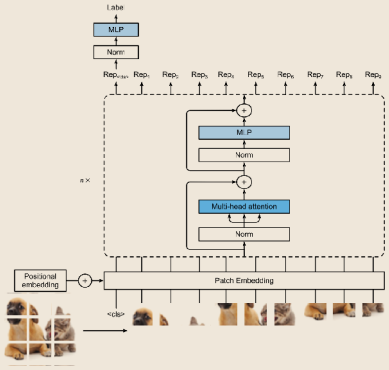
↑ Mục Lục

3.1 💥 [Core — Bread & Butter] Vision Transformer (ViT) & Swin Transformer (`CV15`, `CV16`)

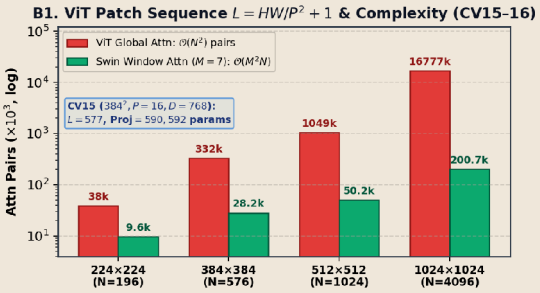
💡 Intuition (Plain English First)

Transformers revolutionized NLP by treating a sentence as a sequence of word tokens and computing all-to-all Self-Attention. Can we feed an image directly into a standard Transformer?

- **Why We Can't Use Individual Pixels as Tokens:** A small 224×224 image has $224 \times 224 = 50,176$ pixels. Self-attention compares every token to every other token, creating an $N \times N$ attention matrix of $50,176^2 \approx$ **2.5** billion entries per layer—completely impossible on GPU memory!
- **How ViT Solves This ("An Image is Worth 16×16 Words", `CV15`):**
  1. Chop the $H \times W \times C$ ($224 \times 224 \times 3$) image into a grid of non-overlapping $P \times P$ (16×16) square tiles called **patches**. That immediately shrinks the sequence length from $50,176$ pixels down to just $\frac{224}{16} \times \frac{224}{16} = 14 \times 14 =$ **196** patches!
  2. Flatten each $16 \times 16 \times 3$ patch into a vector of 768 numbers and multiply by a learnable linear projection matrix E (implemented in practice as a single `Conv2d` with kernel size $P = 16$ and stride $S = 16$).
  3. Prepend one extra learnable `[CLS]` **(classification) token** at the front of the sequence (making the total sequence length $L = N + 1 = 197$), and add a learnable **1D Positional Embedding** to every token so the Transformer knows which patch came from the top-left vs. bottom-right!
- **Inductive Bias & High-Resolution Fine-Tuning (`CV16`):**
  - Unlike CNNs—which hard-code **2D spatial locality** (pixels only talk to immediate neighbors first) and **translation equivariance** (weight sharing across the grid) into every layer—a standard ViT has **almost zero spatial inductive bias** (only the initial patch cut is local; every self-attention layer is global). That's why ViT underperforms ResNets when trained from scratch on small datasets, but scales better on massive datasets ($>$ 10M–100M images).
  - *What happens when you fine-tune a pretrained* 224×224 *ViT on higher-resolution* 384×384 *images?* Keep the patch size $P = 16$ identical! That way, each patch still has $16 \times 16 \times 3 = 768$ values, so the patch projection matrix E and all Transformer weights need **zero modification**. Only the number of patches increases ($196 \to 576$), which we handle by reshaping the 14×14 pretrained patch positional embeddings into a 2D grid and applying **2D bicubic interpolation** to 24×24!
- **Why Swin Transformer Was Invented (`CV16`):** Even with patches, global ViT attention scales **quadratically** $\mathcal{O}(N^2)$ with image area—making it too slow for high-resolution object detection and segmentation (1024×1024). **Swin Transformer** restricts self-attention to small non-overlapping local **Windows of $M \times M$ patches** (e.g., $M = 7$), dropping complexity to **linear** $\mathcal{O}(M^2 N)$ in image size! To let information travel across window borders, every second layer **shifts the window grid** by $\lfloor M/2 \rfloor = 3$ patches (**SW-MSA**), while **Patch Merging** layers (2×2 spatial concatenation + linear channel reduction) build a multi-scale ResNet/FPN-like feature pyramid.



Vision Transformer (ViT) Patch Projection, Class Token & Positional Embeddings



ViT Patch Sequence Length L = HW/P^2 + 1 & Global vs. Windowed Attention Complexity

📐 The Math & Every Symbol Explained

Given an input image $X \in \mathbb{R}^{H \times W \times C}$ (H rows, W columns, C channels) and patch size $P \times P$:

- N: Total number of image patches:

$$N = \frac{H}{P} \times \frac{W}{P} = \frac{H \cdot W}{P^2}$$

- D: Transformer hidden embedding dimension (e.g., $D = 768$ for ViT-Base).
- $E \in \mathbb{R}^{(P^2C) \times D}$: Linear patch projection matrix—an $n \times m$ matrix with $n = P^2C$ rows (flattened patch size) and $m = D$ columns (embedding dimension).

$$\text{Params}_{\text{patch_proj}} = (P^2C) \cdot D + D \quad \text{(with bias)}$$

- $L = N + 1$: Total token sequence length entering the Transformer Encoder after prepending the learnable class token $x_{\text{class}} \in \mathbb{R}^{1 \times D}$ (1 row, D columns).
- $E_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$: Learnable 1D positional embedding matrix—an $n \times m$ matrix with $n = N + 1$ rows and $m = D$ columns, adding $(N + 1)D$ learnable parameters.
- **Single ViT Encoder Block Parameter Count (with bias, MLP expansion ratio $r_{\text{MLP}} = 4$):**
  - **Multi-Head Self-Attention (MHA):** Q, K, V projections $(3 \times (D^2 + D))$ + output projection W_O $(D^2 + D) = \mathbf{4D^2 + 4D}$. *(Independent of head count h because $h \times (D \times \frac{D}{h}) = D^2$!)*
  - **Feed-Forward MLP ($D \to 4D \to D$):** $W_1 \in \mathbb{R}^{D \times 4D}$ $(4D^2 + 4D)$ + $W_2 \in \mathbb{R}^{4D \times D}$ $(4D^2 + D) = \mathbf{8D^2 + 5D}$.
- **Global Self-Attention (ViT) vs. Windowed Self-Attention (Swin, Window Size $M \times M$):**

$$\Omega(\text{Global MSA}) = 4ND^2 + 2N^2D = \mathcal{O}(N^2), \qquad \Omega(\text{W-MSA}) = 4ND^2 + 2M^2ND = \mathcal{O}(M^2N)$$

🔢 Step-by-Step Worked Toy Example

- **Example A (Full ViT-Base Tokenization & Parameter Audit — `CV15`):** Let input image be $H = 224, W = 224, C = 3$, patch size $P = 16$, and hidden dimension $D = 768$.
  1. **Number of Patches (N) & Sequence Length (L):**

$$N = \frac{224 \times 224}{16^2} = 14 \times 14 = \mathbf{196} \text{ patches}, \qquad L = N + 1 = \mathbf{197} \text{ tokens}$$

  2. **Flattened Patch Dimension & Patch Projection Parameters (with bias):**

- Each patch has $P^2C = 16^2 \times 3 = 256 \times 3 = \mathbf{768}$ pixel values.
- Weight matrix E has shape 768×768 plus a 768-dim bias vector:

$$\text{Params}_{\text{patch_proj}} = (768 \times 768) + 768 = 589,824 + 768 = \mathbf{590,592}$$

3. **Class Token & Positional Embedding Parameters:**
   - Class token x_{class} (1×768): 768 parameters.
   - Positional embedding E_{pos} (197×768): $197 \times 768 = \mathbf{151,296}$ parameters.
4. **MHA vs. MLP Parameters in One Transformer Block ($D = 768$, without bias for quick ratio):**
   - MHA $\approx 4D^2 = 4 \times 768^2 = 2,359,296$ ($\frac{1}{3}$ of block parameters).
   - MLP $\approx 8D^2 = 8 \times 768^2 = 4,718,592$ ($\frac{2}{3}$ of block parameters—the MLP holds twice as many weights as self-attention!).

- **Example B (High-Resolution Fine-Tuning & Swin Window Speedup — `CV16`):**
  1. **Fine-tuning ViT-B/16 at $H' = W' = 384$:**
     - Patch size stays $P = 16 \implies$ new patch grid is $\frac{384}{16} \times \frac{384}{16} = 24 \times 24 = \mathbf{576}$ patches, so total sequence length is $L' = 576 + 1 = \mathbf{577}$ tokens.
     - We interpolate the 14×14 patch positional embeddings to 24×24 via 2D bicubic interpolation (leaving the 1×768 `[CLS]` positional embedding untouched).
  2. **Global Attention vs. Swin Window Attention ($M = 7$) on a 56×56 patch grid ($N = 3,136$):**
     - Global ViT pairwise token comparisons: $N^2 = 3,136^2 = \mathbf{9,834,496}$.
     - Swin W-MSA pairwise token comparisons (64 windows of $7 \times 7 = 49$ tokens each): $64 \times 49^2 = M^2N = 49 \times 3,136 = \mathbf{153,664}$—a $64\times$ **reduction** in attention matrix compute!

🎯 Exam Takeaway & Common Traps

- **Trap 1 (`CV15` Don't Forget the `+1` `[CLS]` Token!):** When an exam question asks for the sequence length entering the ViT encoder or the first dimension of the positional embedding matrix E_{pos}, it is $N + 1 = \frac{HW}{P^2} + 1$ (not just N).
- **Trap 2 (`CV15` Multi-Head Parameter Invariance):** Splitting $D = 768$ into $h = 12$ heads of $d_k = 64$ vs. $h = 8$ heads of $d_k = 96$ changes **zero** learnable parameters in MHA!
- **Trap 3 (`CV16` Swin Patch Merging):** A Swin Patch Merging layer groups 2×2 neighboring patches (multiplying channels by 4: $C \to 4C$) and applies a linear layer to reduce $4C \to 2C$—halving spatial resolution $(H/2, W/2)$ while doubling channels $2C$, exactly mirroring a CNN downsampling stage!

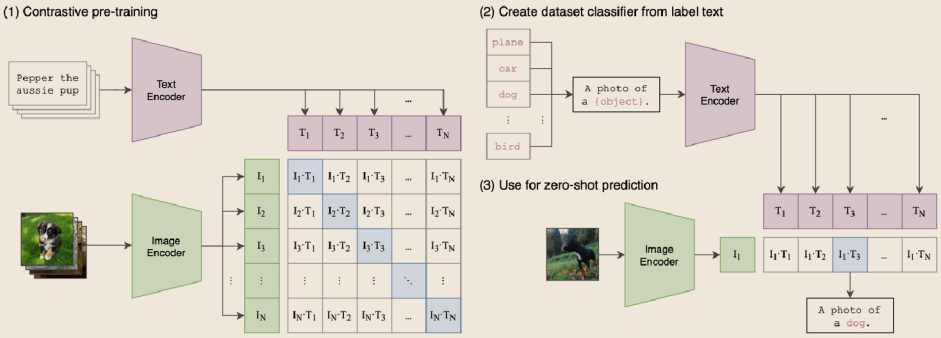
3.2 🌟 [Core — Bread & Butter] CLIP (Contrastive Language-Image Pre-training) & Modern Augmentation/SSL (`CV17`, `CV20`)

💡 Intuition (Plain English First)

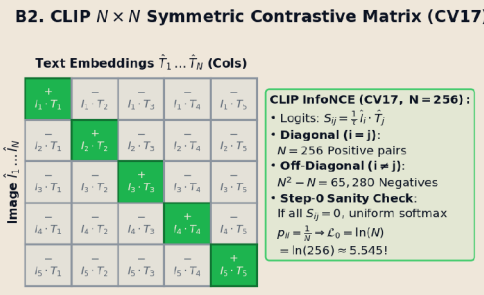
- **CLIP (`CV17`):** Traditional image classifiers train a fixed $1,000$-way linear head on manually labeled classes—meaning they cannot recognize a `'skateboard'` if `'skateboard'` wasn't one of the $1,000$ training indices. **CLIP** learns visual concepts directly from raw internet `(Image, Caption)` pairs! Given a batch of N image-

caption pairs, CLIP runs an Image Encoder and a Text Encoder to get N image vectors and N text vectors, normalizes them to unit length, and computes an $N \times N$ **grid of cosine similarities**. Along the main diagonal ($i = j$) are the N true matching pairs; everywhere off the diagonal ($i \neq j$) are $N^2 - N$ mismatched pairs ("in-batch negatives"). CLIP simply applies Cross-Entropy Loss across every row (picking the right caption for each image) and every column (picking the right image for each caption)! At test time, you can do **zero-shot classification** on any dataset by turning class names into text prompts (`"a photo of a {class}"`) and picking whichever prompt has the highest cosine similarity with the image!

- **Mixup vs. CutMix (`CV20`):**
  - **Mixup** averages two images pixel-by-pixel across the entire canvas ($\tilde{x} = \lambda x_A + (1 - \lambda) x_B$), which looks unnatural and blurry locally.
  - **CutMix** cuts a clean rectangular patch out of image B and pastes it directly over image A, and mixes the one-hot class labels in **exact proportion to the pixel area** of each image ($\lambda = 1 - \frac{\text{patch area}}{\text{image area}}$). Because every local patch still looks like a sharp, natural image, CutMix significantly outperforms Mixup on localization and detection tasks!
- **Masked Autoencoders (MAE, `CV20`):** How did BERT train self-supervised NLP models? By masking 15% of words and predicting them. Why do **Masked Autoencoders (MAE)** mask a whopping 75% **of image patches** instead of 15%? Because images have high spatial redundancy—if you only mask 15%, a model can trivially guess a missing patch by copying the color of its immediate neighbors without learning any semantic object structure! Even better, MAE **completely drops the 75% masked patches from the heavy ViT Encoder**, feeding **only the 25% visible patches** through the encoder (cutting encoder compute and memory by 4×!), and only inserts learnable `[MASK]` tokens right before a tiny lightweight decoder to reconstruct raw pixels via MSE loss.



OpenAI CLIP Contrastive Pre-Training & Zero-Shot Image Classification



CLIP N x N Symmetric Contrastive Similarity Matrix

The Math & Every Symbol Explained

1. **CLIP Symmetric Contrastive (InfoNCE) Loss (`CV17`):** Given a minibatch of N matched image-text pairs $\{(I_i, T_i)\}_{i=1}^{N}$:
   - $\hat{I}_i = \frac{f_I(I_i)}{\|f_I(I_i)\|_2} \in \mathbb{R}^d, \quad \hat{T}_j = \frac{f_T(T_j)}{\|f_T(T_j)\|_2} \in \mathbb{R}^d$: L_2-normalized image and text embeddings.
   - $S \in \mathbb{R}^{N \times N}$ (N rows, N columns): Scaled cosine similarity logit matrix with learnable temperature $\tau > 0$:

     $$S_{ij} = \frac{1}{\tau} \hat{I}_i \cdot \hat{T}_j$$

   - **Number of Positive & Negative Pairs in Batch N:**

     $$\text{Positives} = N \quad (\text{diagonal } i = j), \qquad \text{Negatives} = N^2 - N = N(N-1) \quad (\text{off-diagonal } i \neq j)$$

   - **Symmetric InfoNCE Loss:**

     $$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}\left(\mathcal{L}_{I \to T} + \mathcal{L}_{T \to I}\right) = -\frac{1}{2N} \sum_{i=1}^{N} \left[\ln \frac{\exp(S_{ii})}{\sum_{j=1}^{N} \exp(S_{ij})} + \ln \frac{\exp(S_{ii})}{\sum_{j=1}^{N} \exp(S_{ji})} \right]$$

   - **Step-0 Initialization Sanity Check:** When all logits S_{ij} are equal (e.g., $S_{ij} = c$), every softmax probability is $\frac{1}{N}$, so:

     $$\mathcal{L}_{\text{CLIP}}^{(0)} = -\ln\left(\frac{1}{N}\right) = \ln(N)$$

2. **Mixup & CutMix Label Mixing (`CV20`):**
   - **Mixup:** Given $\lambda \sim \text{Beta}(\alpha, \alpha)$:

     $$\tilde{x} = \lambda x_A + (1-\lambda) x_B, \qquad \tilde{y} = \lambda y_A + (1-\lambda) y_B$$

   - **CutMix:** Given a binary rectangular mask $M \in \{0,1\}^{H \times W}$ (H rows, W columns) of cut-out box dimensions $r_h \times r_w$ where $M = 0$ inside the pasted box and $M = 1$ outside:

     $$\tilde{x} = M \odot x_A + (\mathbf{1} - M) \odot x_B, \qquad \lambda = 1 - \frac{r_w \cdot r_h}{W \cdot H}, \qquad \tilde{y} = \lambda y_A + (1-\lambda) y_B$$

🔢 Step-by-Step Worked Toy Example

- **Example A (CLIP Batch Matrix Audit & Step-0 Loss — `CV17`):**
  1. **Toy Minibatch ($N = 4$):**
     - The similarity matrix S has shape $4 \times 4 = 16$ total entries.
     - **Positive pairs:** $N = \mathbf{4}$ (diagonal entries $S_{11}, S_{22}, S_{33}, S_{44}$).
     - **Negative pairs:** $N(N-1) = 4 \times 3 = \mathbf{12}$ off-diagonal entries.
     - **Step-0 uniform initialization loss:** $\mathcal{L}_0 = \ln(4) \approx \mathbf{1.3863}$.
  2. **Production Minibatch ($N = 256$):**
     - **Positive pairs:** 256.
     - **Negative pairs:** $256^2 - 256 = 65,536 - 256 = \mathbf{65,280}$.
     - **Step-0 uniform initialization loss:** $\mathcal{L}_0 = \ln(256) \approx \mathbf{5.5452}$.

- **Example B (CutMix Label Calculation & MAE Visible Token Count — `CV20`):**
  1. **CutMix Label Vector:** Suppose image x_A (class 0: $[1,0,0]$) and image x_B (class 1: $[0,1,0]$) both have resolution $H \times W = 100 \times 100$ ($10,000$ pixels). We cut a $r_h \times r_w = 40 \times 50$ rectangular patch ($2,000$ pixels) from x_B and paste it onto x_A:

     $$\lambda = 1 - \frac{50 \times 40}{100 \times 100} = 1 - \frac{2,000}{10,000} = 1 - 0.20 = \mathbf{0.80}$$

     $$\tilde{y} = 0.80\,[1,0,0] + 0.20\,[0,1,0] = [\mathbf{0.80},\ \mathbf{0.20},\ \mathbf{0.00}]$$

  2. **MAE Encoder Token Count:** For a 224×224 image with $P = 16$ ($N = 196$ total patches) and masking ratio $m = 75\%$:
     - Only $N_{\text{vis}} = (1 - 0.75) \times 196 = 0.25 \times 196 = \mathbf{49}$ visible patch tokens enter the ViT Encoder!
     - Before the lightweight decoder, we append $196 - 49 = \mathbf{147}$ shared learnable `[MASK]` tokens (plus positional embeddings) to restore the full 196-patch grid and compute MSE reconstruction loss **only on the 147 masked patches**.

🎯 Exam Takeaway & Common Traps

- **Trap 1 (`CV17` CLIP Loss Symmetry):** CLIP's InfoNCE loss is the **average of two N-way Cross-Entropy losses** (image-to-text across rows, and text-to-image across columns), NOT N^2 independent binary cross-entropy losses!
- **Trap 2 (`CV20` MAE Asymmetric Encoder-Decoder):** MAE never feeds `[MASK]` tokens into the heavy ViT Encoder—feeding `[MASK]` tokens into the encoder would waste $4\times$ compute and create a distribution shift between pretraining and fine-tuning!

3.3 🔬 [Core + Deep-Dive] Diffusion Models & Generative Vision: DDPM, DDIM, Latent Diffusion & CFG

💡 Intuition (Plain English First)

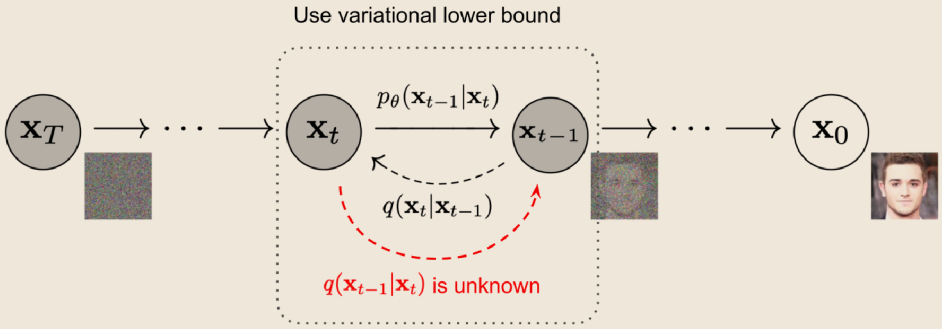
How do modern image generators like DDPM and Stable Diffusion conjure photorealistic images out of pure static noise?

1. **The Forward Process (Destroying an Image with Noise) & The $\mathcal{O}(1)$ Shortcut:**
   - Start with a clean training photo x_0. At each timestep $t = 1, 2, \ldots, T$ (typically $T = 1000$), mix in a tiny bit of fresh Gaussian static noise until, at step T, x_T is indistinguishable from pure random static $\mathcal{N}(0, I)$.
   - *The "Nice Property" ($\mathcal{O}(1)$ Forward Jump):* During training, do we have to run a slow 500-step `for` loop just to generate a noisy image x_{500}? **No!** Because the sum of independent Gaussian random variables is still a single Gaussian, we can jump directly from the clean image x_0 to **any** noisy timestep x_t in **one single formula**:

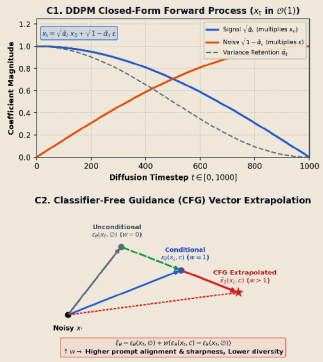
     $$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

     where $\bar{\alpha}_t \in (0, 1)$ measures how much original image signal remains at step t (near 1 when t is small; near 0 when $t \to T$).

2. **The Reverse Process (Why Predict the Noise ϵ Instead of x_0?):**
   - To generate a new image, we start from pure noise $x_T \sim \mathcal{N}(0, I)$ and train a neural network (a time-conditioned U-Net or Diffusion Transformer `DiT`) to peel away a little bit of noise at each step $t \to t-1$.
   - Ho et al. (DDPM) showed that training the network $\epsilon_\theta(x_t, t)$ with simple **Mean Squared Error (MSE) loss to predict the exact noise vector** ϵ that was added to x_0 is vastly more stable than asking it to output the clean image directly, and is mathematically equivalent to **Score Matching** (estimating the gradient of the log data density $\nabla_{x_t} \log q(x_t)$)!
3. **DDPM vs. DDIM (Why 20 Steps Instead of 1000?):**
   - Standard **DDPM** reverse sampling is a stochastic Markov chain that injects fresh random noise $\sigma_t z$ at every step, requiring all $1,000$ sequential steps.
   - **DDIM (Denoising Diffusion Implicit Models)** constructs a **non-Markovian deterministic process** (a probability-flow ODE) that has the **exact same training marginal distribution** $q(x_t \mid x_0)$ as DDPM. That means you can take an **already-trained DDPM checkpoint with ZERO retraining** and sample high-quality images deterministically in just 20 **to** 50 **steps**!
4. **Latent Diffusion (Stable Diffusion) & Classifier-Free Guidance (CFG):**
   - Running diffusion directly in $512 \times 512 \times 3$ pixel space is painfully slow. **Latent Diffusion** first uses a frozen pretrained **VAE Encoder** to compress the image by $f = 8\times$ along height and width into a tiny $64 \times 64 \times 4$ latent tensor z_0—reducing spatial area (and attention tokens) by $8^2 = 64\times$!
   - How does the text prompt `"an astronaut riding a horse"` steer the image? Via **Cross-Attention** inside the U-Net/DiT: the noisy image latent z_t provides the **Queries (Q)**, while the CLIP/T5 text embeddings provide the **Keys (K) and Values (V)**.
   - Finally, **Classifier-Free Guidance (CFG)** makes the image follow the text prompt much more sharply: at each denoising step, we run the network twice—once **with** the text prompt $\epsilon_\theta(z_t, c)$ and once **without** the prompt $\epsilon_\theta(z_t, \varnothing)$—and **extrapolate** in the direction pointing from unconditional to conditional by a guidance weight $w > 1$ (e.g., $w = 7.5$).



DDPM Forward Diffusion q(x_t | x_{t-1}) & Reverse Denoising p_theta(x_{t-1} | x_t) Markov Chain



DDPM Closed-Form Forward Schedule & Classifier-Free Guidance (CFG) Extrapolation

📐 The Math & Every Symbol Explained

- x_0: Clean training image (or clean VAE latent z_0).
- $t \in \{1, 2, \dots, T\}$: Diffusion timestep ($t = 0$ is clean data; $t = T$ is pure isotropic Gaussian noise).
- $\beta_t \in (0, 1)$: Forward variance schedule at step t ($\beta_1 < \beta_2 < \cdots < \beta_T$, e.g., 10^{-4} to 0.02).
- $\alpha_t = 1 - \beta_t$: Single-step signal retention factor.
- $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$: Cumulative signal variance retention from step 0 to step t.
- $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$: Ground-truth standard Gaussian noise sampled with the exact same tensor shape as x_0.
- $\epsilon_\theta(x_t, t, c)$: Neural network (U-Net or DiT with parameters θ) that takes noisy input x_t, timestep t, and conditioning c, and predicts the noise ϵ.

1. **Single-Step & Closed-Form $\mathcal{O}(1)$ Forward Diffusion Process:**

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(x_t;\ \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t \mathbf{I}\right)$$

$$q(x_t \mid x_0) = \mathcal{N}\left(x_t;\ \sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t)\mathbf{I}\right) \quad \Longleftrightarrow \quad x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$$

(Notice that the signal variance weight $\bar{\alpha}_t$ and noise variance weight $1 - \bar{\alpha}_t$ always sum to 1: $(\sqrt{\bar{\alpha}_t})^2 + (\sqrt{1-\bar{\alpha}_t})^2 = 1$, preserving unit variance if x_0 has unit variance!)

2. **Simplified Noise-Prediction Training Loss (DDPM):**

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[1,T],\, x_0,\, \epsilon \sim \mathcal{N}(\mathbf{0},\mathbf{I})}\left[\left\| \epsilon - \epsilon_\theta\left(\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon,\ t\right)\right\|_2^2\right]$$

3. **Classifier-Free Guidance (CFG) Extrapolation Formula:** Let $\epsilon_\theta(z_t, c)$ be the conditional noise prediction (given text prompt c), $\epsilon_\theta(z_t, \varnothing)$ be the unconditional noise prediction (given an empty/null token $\varnothing$, trained by randomly dropping the prompt 10–20% of the time during training), and $w \geq 1$ be the guidance scale:

$$\tilde{\epsilon}_\theta(z_t, c) = \epsilon_\theta(z_t, \varnothing) + w \cdot (\epsilon_\theta(z_t, c) - \epsilon_\theta(z_t, \varnothing)) = w\, \epsilon_\theta(z_t, c) + (1 - w)\, \epsilon_\theta(z_t, \varnothing)$$

🔢 Step-by-Step Worked Toy Example

- **Example A (Closed-Form DDPM Forward Noise Calculation at Step t):** Suppose at timestep $t = 400$, the cumulative signal product is $\bar{\alpha}_{400} = 0.36$. For a single pixel with clean value $x_0 = 2.0$ and sampled Gaussian

noise $\epsilon = -0.5$, what are the mean, variance, and exact noisy value x_{400}?

1. **Signal coefficient:** $\sqrt{\bar{\alpha}_t} = \sqrt{0.36} = \mathbf{0.60}$.
2. **Noise standard deviation:** $\sqrt{1 - \bar{\alpha}_t} = \sqrt{1 - 0.36} = \sqrt{0.64} = \mathbf{0.80}$ (variance is $1 - \bar{\alpha}_t = 0.64$).
3. **Exact noisy pixel x_{400}:**

   $$x_{400} = (0.60)(2.0) + (0.80)(-0.5) = 1.20 - 0.40 = \mathbf{0.80}$$

   (And if we know $x_{400} = 0.80$ and our network perfectly predicts $\hat{\epsilon} = -0.5$, we can immediately recover the clean pixel estimate: $\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\epsilon}}{\sqrt{\bar{\alpha}_t}} = \frac{0.80 - 0.80(-0.5)}{0.60} = \frac{1.20}{0.60} = 2.0$!)

- **Example B (Classifier-Free Guidance & Latent Diffusion Compression):**
  1. **CFG Extrapolation:** At step t, suppose the unconditional noise prediction for a latent coordinate is $\epsilon_\theta(z_t, \varnothing) = 0.20$, the text-conditioned prediction is $\epsilon_\theta(z_t, c) = 0.60$, and the user sets guidance scale $w = 7.5$:

     $$\tilde{\epsilon} = 0.20 + 7.5 \times (0.60 - 0.20) = 0.20 + 7.5 \times 0.40 = 0.20 + 3.00 = \mathbf{3.20}$$

     (Check when $w = 1.0$: $\tilde{\epsilon} = 0.20 + 1.0(0.40) = 0.60$, which is standard conditional generation without extra guidance; when $w = 0$: $\tilde{\epsilon} = 0.20$, which is purely unconditional!)
  2. **Latent Diffusion Spatial Token Reduction:** A $512 \times 512 \times 3$ image has $512 \times 512 = 262,144$ spatial pixels. With a VAE downsampling factor of $f = 8$, the latent z_0 has spatial grid $64 \times 64 = 4,096$ locations —an exact $8^2 = \mathbf{64}\times$ reduction in spatial positions (and a $64^2 = 4,096\times$ **reduction** in all-to-all self-attention matrix size!).

🎯 Exam Takeaway & Common Traps

- **Trap 1 (Forward Variance Sum):** In $x_t = \sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1 - \bar{\alpha}_t}\,\epsilon$, remember the square roots! The **standard deviations** are $\sqrt{\bar{\alpha}_t}$ and $\sqrt{1 - \bar{\alpha}_t}$; the **variances** are $\bar{\alpha}_t$ and $1 - \bar{\alpha}_t$.
- **Trap 2 (DDIM Retraining Myth):** DDIM does **NOT** require retraining a DDPM model—it uses the exact same trained noise-predictor weights $\epsilon_\theta(x_t, t)$ with a deterministic non-Markovian update rule.
- **Trap 3 (Cross-Attention Q, K, V Sources in Text-to-Image Diffusion):** In Stable Diffusion's Cross-Attention layers, **Queries (Q)** always come from the **noisy image latent** z_t (so the output keeps the image's spatial $H \times W$ shape), while **Keys (K) and Values (V)** come from the **text prompt embeddings** c!

▼ 📌 Optional / Stage-2 Interview Bonus (Skip for Timed MCQ): Full DDPM, Score-Matching, DDIM & CFG Mathematical Derivations

1. **Why $q(x_t \mid x_0)$ Collapses in Closed Form (Reparameterization Induction):** Since $x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \epsilon_{t-1}$ and $x_{t-1} = \sqrt{\alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_{t-1}}\, \epsilon_{t-2}$, substituting recursively gives:

$$x_t = \sqrt{\alpha_t \alpha_{t-1}}\, x_{t-2} + \underbrace{\sqrt{\alpha_t(1-\alpha_{t-1})}\, \epsilon_{t-2} + \sqrt{1-\alpha_t}\, \epsilon_{t-1}}_{\text{Sum of two independent Gaussians } \sim \mathcal{N}(0,\, (1-\alpha_t\alpha_{t-1})\mathbf{I})}$$

   By induction down to $t = 0$ with $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, we obtain $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$.

2. **Exact Equivalence Between Noise Prediction ϵ_θ and Score Matching $\nabla_{x_t} \log q(x_t)$:** For $q(x_t \mid x_0) = \mathcal{N}(x_t;\, \sqrt{\bar{\alpha}_t} x_0,\, (1-\bar{\alpha}_t)\mathbf{I})$, taking the gradient of the log-density with respect to x_t yields:

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t}\, x_0}{1-\bar{\alpha}_t} = -\frac{\sqrt{1-\bar{\alpha}_t}\, \epsilon}{1-\bar{\alpha}_t} = -\frac{\epsilon}{\sqrt{1-\bar{\alpha}_t}} \implies \epsilon_\theta(x_t, t) \approx -\sqrt{1-\bar{\alpha}_t}\, \nabla_{x_t} \log q(x_t)$$

3. **Why Classifier-Free Guidance Works (Implicit Classifier Derivation):** By Bayes' rule on the score of an implicit classifier $p(c \mid z_t) \propto \frac{p(z_t \mid c)}{p(z_t)}$:

$$\nabla_{z_t} \log p(c \mid z_t) = \nabla_{z_t} \log p(z_t \mid c) - \nabla_{z_t} \log p(z_t) \propto -\left(\epsilon_\theta(z_t, c) - \epsilon_\theta(z_t, \varnothing)\right)$$

   Therefore, sampling with $\tilde{\epsilon}_\theta(z_t, c) = \epsilon_\theta(z_t, \varnothing) + w(\epsilon_\theta(z_t, c) - \epsilon_\theta(z_t, \varnothing))$ is equivalent to following the score of the sharpened distribution $\tilde{p}(z_t \mid c) \propto p(z_t) \cdot p(c \mid z_t)^w$ without needing a separate noisy-image classifier!

